

Bengaluru_House_Analysis

October 9, 2025

1 Shrihari Viswanathan

Data analysis of the kaggle data set <https://www.kaggle.com/datasets/bhaweshsinha07/bengaluru-house-price-dataset>

```
[1]: import pandas as pd
import numpy as np
import re
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split, StratifiedKFold, \
    cross_val_score, cross_val_predict, GridSearchCV
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.tree import DecisionTreeRegressor
from xgboost import XGBRegressor
from sklearn.linear_model import Ridge, Lasso
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from sklearn.preprocessing import PolynomialFeatures
%matplotlib inline
```

```
[2]: df = pd.read_csv("Bengaluru_House_Data.csv")
```

```
[3]: df.head()
```

```
[3]:
```

		area_type	availability	location	size	\
0	Super built-up	Area	19-Dec	Electronic City Phase II	2 BHK	
1	Plot	Area	Ready To Move	Chikka Tirupathi	4 Bedroom	
2	Built-up	Area	Ready To Move	Uttarahalli	3 BHK	
3	Super built-up	Area	Ready To Move	Lingadheeranahalli	3 BHK	
4	Super built-up	Area	Ready To Move	Kothanur	2 BHK	

	society	total_sqft	bath	balcony	price
0	Coomee	1056	2.0	1.0	39.07
1	Theanmp	2600	5.0	3.0	120.00
2	NaN	1440	2.0	3.0	62.00
3	Soiewre	1521	3.0	1.0	95.00

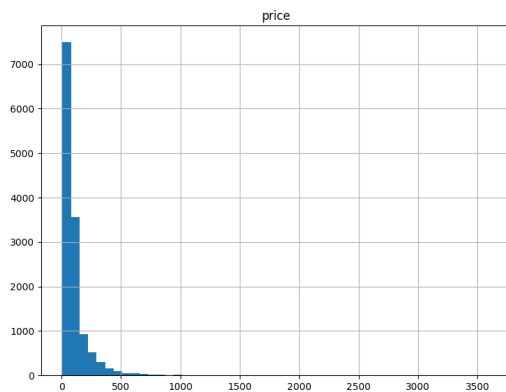
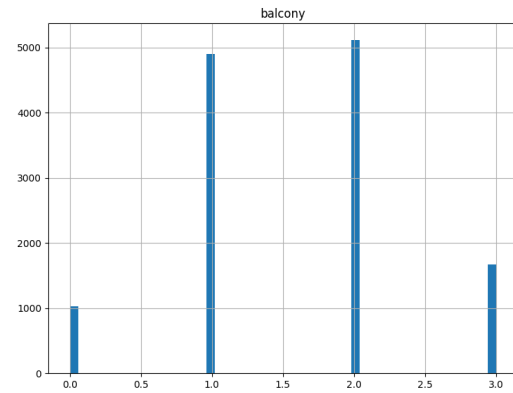
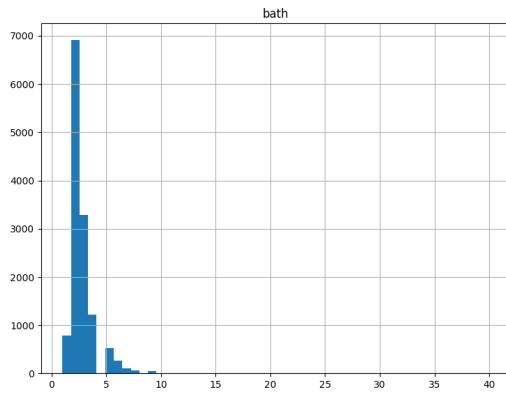
```
4      NaN      1200    2.0      1.0    51.00
```

```
[4]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 13320 entries, 0 to 13319
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  -
0   area_type       13320 non-null  object
1   availability     13320 non-null  object
2   location        13319 non-null  object
3   size            13304 non-null  object
4   society         7818 non-null   object
5   total_sqft      13320 non-null  object
6   bath            13247 non-null  float64
7   balcony         12711 non-null  float64
8   price           13320 non-null  float64
dtypes: float64(3), object(6)
memory usage: 936.7+ KB
```

```
[5]: df.hist(bins = 50, figsize = (20, 15))
```

```
[5]: array([[<Axes: title={'center': 'bath'}>,
          <Axes: title={'center': 'balcony'}>]],
          [<Axes: title={'center': 'price'}>, <Axes: >]], dtype=object)
```



The null values are there in location, society, bath, balcony

Updating “bath” and “balcony” to their respective means

```
[6]: df["bath"] = df["bath"].fillna(df["bath"].mean())
```

```
[7]: print(df["balcony"].isnull().sum())
df["balcony"] = df["balcony"].fillna(df["balcony"].mean())
```

609

Assigning the null values to the most common location and

Filtering out <2 locations from “location” to others

```
[8]: df["location"].value_counts()
```

```
[8]: location
Whitefield                540
Sarjapur Road             399
Electronic City           302
Kanakpura Road            273
```

```

Thanisandra                234
...
3rd Stage Raja Rajeshwari Nagar    1
Chuchangatta Colony              1
Electronic City Phase 1,          1
Chikbasavanapura                 1
Abshot Layout                    1
Name: count, Length: 1305, dtype: int64

```

```

[9]: df["location"] = df["location"].fillna("Whitefield")

counts = df["location"].value_counts()
df["location"] = df["location"].apply(lambda x : x if counts[x] > 2 else
↳ "others")

```

Removing the society column as more than half of the data is missing

```

[10]: df.drop(["society"], axis = 1, inplace = True)

```

taking care of the values in the size columns

```

[11]: df["size"].unique()

```

```

[11]: array(['2 BHK', '4 Bedroom', '3 BHK', '4 BHK', '6 Bedroom', '3 Bedroom',
'1 BHK', '1 RK', '1 Bedroom', '8 Bedroom', '2 Bedroom',
'7 Bedroom', '5 BHK', '7 BHK', '6 BHK', '5 Bedroom', '11 BHK',
'9 BHK', nan, '9 Bedroom', '27 BHK', '10 Bedroom', '11 Bedroom',
'10 BHK', '19 BHK', '16 BHK', '43 Bedroom', '14 BHK', '8 BHK',
'12 Bedroom', '13 BHK', '18 Bedroom'], dtype=object)

```

```

[12]: pattern = [r"[1-9]\s*BHK", r"[1-9]\s*Bedroom", r"[1-9]\s*RK"]

def unwanted_text_remover(column):
    for index, val in column.items():
        if isinstance(val, str) and (re.search(pattern[0], val)):
            number = re.search(r"[1-9]", val).group()

            df.loc[index, "Bedroom"] = number
            df.loc[index, ["Hall", "Kitchen"]] = 1

        if isinstance(val, str) and (re.search(pattern[1], val)):
            number = re.search(r"[1-9]", val).group()

            df.loc[index, "Bedroom"] = number
            df.loc[index, ["Hall", "Kitchen"]] = 0

        if isinstance(val, str) and re.search(pattern[2], val):
            number = re.search(r"[1-9]", val).group()
            df.loc[index, "Bedroom"] = number

```

```
df.loc[index, "Kitchen"] = 1
df.loc[index, "Hall"] = 0
```

```
unwanted_text_remover(df["size"])
```

```
[13]: df.drop(["size"], axis = 1, inplace = True)
```

```
[14]: df[["Bedroom", "Hall", "Kitchen"]].isnull().sum()
```

```
[14]: Bedroom    30
      Hall       30
      Kitchen   30
      dtype: int64
```

```
[15]: print(df[df["Bedroom"].isnull()][["Bedroom", "Hall", "Kitchen"]])
```

	Bedroom	Hall	Kitchen
579	NaN	NaN	NaN
1735	NaN	NaN	NaN
1775	NaN	NaN	NaN
1795	NaN	NaN	NaN
2264	NaN	NaN	NaN
2809	NaN	NaN	NaN
2862	NaN	NaN	NaN
3096	NaN	NaN	NaN
3523	NaN	NaN	NaN
4510	NaN	NaN	NaN
4691	NaN	NaN	NaN
5333	NaN	NaN	NaN
6423	NaN	NaN	NaN
6451	NaN	NaN	NaN
6636	NaN	NaN	NaN
6719	NaN	NaN	NaN
7680	NaN	NaN	NaN
8306	NaN	NaN	NaN
8565	NaN	NaN	NaN
8636	NaN	NaN	NaN
8703	NaN	NaN	NaN
9511	NaN	NaN	NaN
9769	NaN	NaN	NaN
10634	NaN	NaN	NaN
11019	NaN	NaN	NaN
11128	NaN	NaN	NaN
11142	NaN	NaN	NaN
11327	NaN	NaN	NaN
11569	NaN	NaN	NaN
13067	NaN	NaN	NaN

```
[16]: for col in ["Bedroom", "Hall", "Kitchen"]:
        df[col] = df[col].astype(float)
        df[col] = df[col].fillna(df[col].mean()).astype(int)
```

```
[17]: df.isnull().sum()
```

```
[17]: area_type      0
      availability  0
      location      0
      total_sqft    0
      bath          0
      balcony       0
      price         0
      Bedroom       0
      Hall          0
      Kitchen       0
      dtype: int64
```

Area Type can be one hot encoderised”

```
[18]: df["area_type"].value_counts()
```

```
[18]: area_type
      Super built-up Area    8790
      Built-up Area       2418
      Plot Area           2025
      Carpet Area          87
      Name: count, dtype: int64
```

```
[19]: dummy = pd.get_dummies(df["area_type"])
      df = pd.concat([df, dummy], axis =1)
      df.head()
```

```
[19]:
```

		area_type	availability	location	total_sqft	\
0	Super built-up	Area	19-Dec	Electronic City Phase II	1056	
1	Plot	Area	Ready To Move	Chikka Tirupathi	2600	
2	Built-up	Area	Ready To Move	Uttarahalli	1440	
3	Super built-up	Area	Ready To Move	Lingadheeranahalli	1521	
4	Super built-up	Area	Ready To Move	Kothanur	1200	

	bath	balcony	price	Bedroom	Hall	Kitchen	Built-up	Area	\
0	2.0	1.0	39.07	2	1	1	False		
1	5.0	3.0	120.00	4	0	0	False		
2	2.0	3.0	62.00	3	1	1	True		
3	3.0	1.0	95.00	3	1	1	False		
4	2.0	1.0	51.00	2	1	1	False		

	Carpet	Area	Plot	Area	Super built-up	Area
--	--------	------	------	------	----------------	------

0	False	False	True
1	False	True	False
2	False	False	False
3	False	False	True
4	False	False	True

```
[20]: df.drop("area_type", axis = 1, inplace = True)
```

The availability can be grouped into their own respective months for regression

```
[21]: for index, val in df["availability"].items():
        if val[3: ] in ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"]:
            df.loc[index, "availability"] = val[3 : ]

df.head()
```

```
[21]:      availability      location  total_sqft  bath  balcony  price \
0          Dec  Electronic City Phase II    1056    2.0      1.0   39.07
1  Ready To Move      Chikka Tirupathi    2600    5.0      3.0  120.00
2  Ready To Move      Uttarahalli    1440    2.0      3.0   62.00
3  Ready To Move  Lingadheeranahalli    1521    3.0      1.0   95.00
4  Ready To Move      Kothanur    1200    2.0      1.0   51.00
```

	Bedroom	Hall	Kitchen	Built-up	Area	Carpet	Area	Plot	Area	\
0	2	1	1	False	False	False	False	False	False	
1	4	0	0	False	False	False	True	True	True	
2	3	1	1	True	False	False	False	False	False	
3	3	1	1	False	False	False	False	False	False	
4	2	1	1	False	False	False	False	False	False	

	Super built-up	Area
0	True	
1	False	
2	False	
3	True	
4	True	

```
[22]: df["availability"].unique()
```

```
[22]: array(['Dec', 'Ready To Move', 'May', 'Feb', 'Nov', 'Oct', 'Sep', 'Mar',
        'Apr', 'Aug', 'Jun', 'Jan', 'Jul', 'Immediate Possession'],
        dtype=object)
```

```
[23]: dummy = pd.get_dummies(df["availability"])
df = pd.concat([df, dummy], axis = 1)
df.drop("availability", axis =1, inplace = True)
```

location is still off, there are too many locations, grouping locations based on their proximity to each other

```
[24]: df["location"].unique()
```

```
[24]: array(['Electronic City Phase II', 'Chikka Tirupathi', 'Uttarahalli',  
        'Lingadheeranahalli', 'Kothanur', 'Whitefield', 'Old Airport Road',  
        'Rajaji Nagar', 'Marathahalli', 'others', '7th Phase JP Nagar',  
        'Gottigere', 'Sarjapur', 'Mysore Road', 'Bisuvanahalli',  
        'Raja Rajeshwari Nagar', 'Ramakrishnappa Layout',  
        'Manayata Tech Park', 'Kengeri', 'Binny Pete', 'Thanisandra',  
        'Bellandur', 'Thanisandra', 'Mangammanapalya', 'Electronic City',  
        'Ramagondanahalli', 'Yelahanka', 'Hebbal', 'Kasturi Nagar',  
        'Kanakpura Road', 'Electronics City Phase 1', 'Kundalahalli',  
        'Chikkalasandra', 'Murugeshpalya', 'Sarjapur Road', 'Ganga Nagar',  
        'HSR Layout', 'Doddathoguru', 'KR Puram', 'Himagiri Meadows',  
        'Bhoganahalli', 'Lakshminarayana Pura', 'Begur Road', 'Devanahalli',  
        'Varthur', 'Bommanahalli', 'Gunjur', 'Devarachikkanahalli',  
        'Hegde Nagar', 'Haralur Road', 'Hennur Road', 'Kothannur',  
        'Kalena Agrahara', 'Cholanayakanahalli', 'Kaval Byrasandra',  
        'ISRO Layout', 'Garudachar Palya', 'EPIP Zone', 'Dasanapura',  
        'Kasavanahalli', 'Sanjay nagar', 'Mysore Highway', 'Domlur',  
        'Sarjapura - Attibele Road', 'Devasthanagalu', 'T Dasarahalli',  
        'Yeshwanthpur', 'Chandapura', 'Green View Layout',  
        'Shantiniketan Layout', 'Peenya', 'Nagarbhavi', 'Jalahalli West',  
        'Byatarayanapura', 'Ramamurthy Nagar', 'Malleswaram',  
        'Akshaya Nagar', 'Shampura', 'Kadugodi', 'LB Shastri Nagar',  
        'Vajarahalli', 'Hormavu', 'Vishwapriya Layout', 'Kudlu Gate',  
        '8th Phase JP Nagar', 'Bommasandra Industrial Area',  
        'Chandra Layout', 'Anandapura', 'Vishveshwarya Layout',  
        'Kengeri Satellite Town', 'Basavanapura', 'Dinnur', 'Kannamangala',  
        'Devarachikkanahalli', 'Sanjeevini Nagar', 'Hulimavu',  
        'Mahalakshmi Layout', 'Hosa Road', 'Keshava Nagar',  
        'RMV Extension', 'Tejaswini Nagar', 'Kothnur Narayanapura',  
        'Attibele', 'CV Raman Nagar', 'Kumaraswami Layout', 'Nagavara',  
        'Hebbal Kempapura', 'Vijayanagar', '8th block Koramangala',  
        'Jakkuru Layout', 'Pattandur Agrahara', 'Ejipura', 'MS Pallya',  
        'Nagasandra', 'Langford Town', 'Kogilu', 'Panathur',  
        'Nagondanahalli', 'Padmanabhanagar', '1st Block Jayanagar',  
        'Kammasandra', 'Off Sarjapur Road', 'Tala Cauvery Layout',  
        'Dasarahalli', 'Magadi Road', 'Ngef Layout', 'Koramangala',  
        'Dommasandra', 'Budigere', 'Dodda Nekkundi Extension',  
        'Mylasandra', 'Kalyan nagar', 'Bank Of Baroda Colony',  
        'Ashwath Nagar', 'Meenakunte', 'OMBR Layout', 'Anand Nagar',  
        'Shree Ananth Nagar Layout', 'Kundalahalli Colony',  
        'Horamavu Agara', 'MCECHS layout', 'Coffee Board Layout',  
        'Ambedkar Nagar', 'KUDLU MAIN ROAD', 'Talaghattapura', 'Balagere',  
        'Jigani', 'Gollarapalya Hosahalli', 'Srirampura',
```


'Old Madras Road', 'Kaggadasapura', 'Silver Springs Layout',
 '9th Phase JP Nagar', 'Jakkur', 'TC Palaya', 'Maruthi Sevanagar',
 'RMV 2nd Stage', 'Giri Nagar', 'Singasandra', 'AECS Layout',
 'Mallasandra', 'Begur', 'JP Nagar', 'Malleshpalya', 'Kodathi',
 'Munnekollal', 'Motappa Layout', 'Akshaya Vana', 'Kaggalipura',
 'Basavanna Nagar', '6th Phase JP Nagar', 'Ulsoor',
 'Thigalarapalya', 'Somasundara Palya', 'T.C PALYA',
 'Basaveshwara Nagar', 'Bommasandra', 'Prashanth Nagar',
 'Ardendale', 'Harlur', 'RMV', 'Bagalur Main Road', 'Kodihalli',
 'Narayanapura', 'Bannerghatta Road', 'Hennur',
 '5th Phase JP Nagar', 'Kodigehaali', 'Gopalapura', 'Billekahalli',
 'Jalahalli', 'Mahadevpura', 'Anekal', 'Hoodi Circle', 'Sompura',
 'Ashirvad Colony', 'Dodda Nekkundi', 'Hosur Road', 'Battarahalli',
 'Gokula Extension', 'Kamakshipalya', 'Sultan Palaya',
 'Sanne Amanikere', 'Ambalipura', 'Hoodi', 'Samethanahalli',
 'Brookefield', 'Yelenahalli', 'Richmond Town', 'Suddaguntepalya',
 'Udayapur Village', 'Vittasandra', 'Thippasandra', 'Hoysalanagar',
 'Defence Colony', '2nd Stage Nagarbhavi', 'Vidyaranyapura',
 'Amruthahalli', 'Kodigehalli', 'NS Palya', 'Subramanyapura',
 'Basavangudi', 'Kenchenahalli', '3rd Block Hrbr Layout',
 'Surabhi Layout', 'Banjara Layout', 'Kereguddadahalli',
 'Maragondanahalli', 'Silk Board', 'Kada Agrahara', 'Kambipura',
 'BHEL Layout', 'Medahalli', 'Banashankari Stage III',
 'Kashi Nagar', 'Sector 7 HSR Layout', 'VHBCS Layout', 'Ittamadu',
 'Rajiv Nagar', 'Gattahalli', 'Arekere', 'Ramanjaneyanagar',
 'Mico Layout', '7th Block Jayanagar', '2nd Phase JP Nagar',
 'Kammanahalli', 'Banashankari', 'Herohalli', 'Chikkabanavar',
 'HRBR Layout', 'Ashwini layout', 'Dooravani Nagar', 'JCR Layout',
 'Nehru Nagar', 'Kanakapura', 'Venkatapura', 'Konanakunte',
 'Hosapalya', 'Doddagubbi', 'Sector 6 HSR Layout',
 'Margondanahalli', 'ITI Layout', 'R.T. Nagar', 'Tumkur Road',
 'Vasanthapura', 'GM Palaya', 'Jalahalli East',
 'Poornapragna Housing Society Layout', 'Hosakerehalli', 'Huskur',
 'Indira Nagar', 'Shanthala Nagar', 'Bettahalsoor',
 'Srinivasa Nagar', 'Byrathi Village', 'Kodichikkanahalli',
 'Ambedkar Colony', 'Varthur Road', 'Anjappa Layout', 'Avalahalli',
 'Sadaramangala', 'Anjanapura', 'Abbigere', 'Tindlu',
 'B Narayanapura', 'Gubbalala', 'Dairy Circle',
 'Parappana Agrahara', 'Jeevan bima nagar',
 'Narayana Nagar 1st Block', 'KSRTC Layout', 'Cunningham Road',
 'New Gurappana Palya', 'Chelekare', 'Palanahalli', 'Vadarpalya',
 'Kudlu', 'Banashankari Stage VI', 'HMT Layout', 'Basavanagara',
 'Cox Town', 'Kathriguppe', 'HBR Layout', 'Yelahanka New Town',
 'Sahakara Nagar', 'Rachenahalli', 'Belathur', 'Sadashiva Nagar',
 'Yelachenahalli', 'Kodbisanhalli', 'Sathya Sai Layout',
 'Vishwanatha Nagenahalli', 'Singapura Village', 'Ferrar Nagar',
 'Green Glen Layout', 'Thubarahalli', 'Naganathapura',

'Horamavu Banaswadi', '1st Phase JP Nagar', 'Kaverappa Layout',
 'NGR Layout', 'Divya Unnathi Layout', 'Vasantha Vallabha Nagar',
 'Seegehalli', 'BEML Layout', 'Alur', 'NRI Layout', 'Sidedahalli',
 'ITPL', 'Thurahalli', 'Kattigenahalli', 'Weavers Colony',
 'Raghavendra Nagar', 'KR Layout', 'Babusapalaya',
 'Nagappa Reddy Layout', 'BTM 1st Stage', 'Hessarghatta',
 'Suraksha Nagar', 'Lakshmi Layout', 'Iblur Village',
 'Kadugondanahalli', 'Basava Nagar', '2nd Block Hrbr Layout',
 'Ananth Nagar', 'Canara Bank Layout', 'Manjunath Nagar',
 'Basapura', 'Channasandra', 'Singena Agrahara',
 'Sector 1 HSR Layout', 'Kacharakanahalli', 'Gopalkrishna Nagar',
 'Choodasandra', 'Sai Gardens', 'Hanumanth Nagar', 'Kaikondrahalli',
 'RWF West Colony', 'Bileshivale', 'Neeladri Nagar', 'Frazer Town',
 'Cooke Town', 'Doddakallasandra', 'Chamrajpet', 'VGP Layout',
 'Vasanth nagar', 'Rayasandra', 'Kalkere', 'Siddapura', 'Makali',
 '5th Block Hbr Layout', 'Pai Layout', 'Roopena Agrahara',
 'Banashankari Stage V', 'Gollahalli', 'Sonnenahalli',
 'Seetharampalya', 'Devanahalli Int. Airport', 'Puttanahalli',
 'Benson Town', '2nd Phase Judicial Layout',
 '1st Block HRBR Layout', 'Sarjapur Road', 'Byadarahalli',
 'Poorna Pragna Layout', 'Judicial Layout', 'Chikka Banaswadi',
 'Outer Ring Road East', 'Kavika Layout', 'Vinayaka Nagar',
 'Banashankari Stage II', 'AMS Layout', '9th Block Jayanagar',
 'Karuna Nagar', 'Ashwathnagar', 'AGS Layout', 'Srinagar',
 'Bannerghatta', 'Immadihalli', 'Banagiri Nagar', 'Hiremath Layout',
 'Jnanabharathi Layout', 'Chikkasandra', '3rd Block Jayanagar',
 'Konanakunte Cross', 'Abbaiah Reddy Layout', 'Tavarekere',
 'Thirumenahalli', 'HOSUR MAIN ROAD', 'Kumara Park',
 'Daadys Gaarden Layout', 'Maruthi Nagar', 'Marsur', 'Kallumantapa',
 'Rustam Bagh Layout', 'New Thippasandra', 'Kumbena Agrahara',
 'Akshayanagara East', 'Doddanekundi', 'Venkatadri Layout',
 'Bommenahalli', 'Vittal Nagar', '3rd Block Banashankari',
 'Bhuvaneshwari Nagar', 'Laggere', 'Victoria Layout',
 'Prithvi Layout', '5th Stage BEML Layout', 'BSM Extension',
 'Banaswadi', 'P&T Layout', 'Byrasandra', 'Veerannapalya',
 'Mathikere Extension', 'Sector 2 HSR Layout', 'Anantapura',
 'Shivaji Nagar', 'Badavala Nagar', '4th Block Koramangala',
 'Belatur', 'Kamakya Layout', 'Vignana Nagar', 'Nallurhalli',
 'Chinnapanahalli', 'Lingarajapuram', 'Laxmi Sagar Layout',
 'Nelamangala', 'Nagavarapalya', 'Hongasandra', 'Harappanahalli',
 'BTM Layout', 'Chennamma Kere', 'Kanaka Nagar', 'Tasker Town',
 'Govindpura', 'Shettigere', 'BEL Road', 'Byagadadhenahalli',
 'Doddabommasandra', 'Tunganagara', 'BTM 2nd Stage',
 'Jakkur Plantation', 'Bikasipura', 'Basaveshwara Nagar Yelahanka',
 '1st Block Koramangala', 'Hoskote', 'Srirampuram', 'Yemlur',
 'Gkvk Layout', 'Doopanahalli', 'Doddaballapur', 'Kaveri Nagar',
 'Alfa Garden Layout', 'Gowdanapalya', 'Gunjur Palya',

'Upkar Layout', 'Sunkadakatte', 'Channasandra Layout',
 'Doctors Layout', 'Austin Town', 'Vishwapriya Nagar',
 'Sarakki Nagar', 'Lottegolla Halli', 'Nandini Layout',
 'B Channasandra', 'Dollars Colony', 'Arehalli', 'Hennur Gardens',
 'Thyagaraja Nagar', 'Bharathi Nagar', 'Cambridge Layout',
 'Doddakannelli', 'Thanisandra Main Road, ', 'Jayanagar',
 'Amruthnagar', 'Palace Guttahalli', 'Akshya Nagar', 'BCC Layout',
 'Mathikere', 'Sri Balaji Krupa Layout', 'Bidadi',
 'Dodsworth Layout', 'Rajiv Gandhi Nagar', 'Kurubarahalli',
 'Bhagyalakshmi Avenue', 'Kengeri Hobli', 'HAL 2nd Stage',
 'Devarabeesana Halli', '1st Stage Indira Nagar', 'Adityanagar',
 'Esther Enclave Layout', 'Volagerekallahalli', 'Shikaripalya',
 '4th T block Jayanagar', 'Nayandanahalli', 'Hagadur',
 'Pattanagere', 'Dollars Layout', 'Bhuvaneswari Nagar', 'Varanasi',
 'Virat Nagar', 'Nobo Nagar', 'Kadubeesanahalli', 'Nanjappa Layout',
 'Dodda Banaswadi', 'Bellari Road', 'Tata Nagar', 'Tirumanahalli',
 'Raghuvanahalli', 'Geddalahalli', 'Brindavan Nagar',
 'Domlur Layout', 'Vijaya Bank Layout', 'Wilson Garden',
 'Nyanappana Halli', 'KR Garden', 'Mallathahalli', 'Bagalur',
 'Vinayak Nagar', 'Crimson Layout', 'Hanumantha Nagar',
 'Bagalakunte', 'Lavelle Road', 'Rajankunte', 'Nagadevanahalli',
 'Madiwala', 'Rajarajeshwari Nagara', 'Munivenkatppa Layout',
 'Kamala Nagar', 'Kumarapalli', 'Nagawara Junction',
 'Maithri Layout', 'Raja Rajeshwari Nagar 5th Stage',
 'Kenchenhalli', 'Kachanayakanahalli', 'Nagaraja Garden',
 '6th block Koramangala', '4th Block Jayanagar', 'Kuvempu Nagar',
 'Canara Bank Colony', 'Seshadripuram', 'Chikkathoguru',
 'Sarvabhouna Nagar', 'SMV layout',
 'Doddanakundi Industrial Area 2', 'OLd Gurappanapalya',
 'Chikkadunnasandra', 'Cottonpet', 'Sampangi Rama Nagar',
 'Pragathi Nagar', 'Ramesh Nagar', 'A Narayanapura', 'Amblipura',
 'Rajarajeshwari nagar', 'Jinkethimmanahalli', 'Vimanapura',
 'Venugopal Reddy Layout', 'Lake City', 'Lakshmipura', 'Kithaganur',
 'J C Nagar', 'Doddakammanahalli', 'Richards Town',
 'Manorayana Palya', 'Mahalakshmi Puram', 'Attur Layout',
 'Brooke Bond First Cross', 'Glass Factory Layout',
 'Yeshwanthpur Industrial Suburb', 'Thanisandra main road',
 'Doddabanahalli', 'D Group Employees Layout',
 'Hosahalli Extension', '4th Phase JP Nagar', '2nd Block Jayanagar',
 'Singanayakanahalli', 'Udaya Nagar', 'Vaishnavi Layout',
 'Ilyas Nagar', '3rd Phase JP Nagar', 'Reliaable Tranquil Layout',
 '8th Block Jayanagar', 'Atmananda Colony', 'Baba Nagar',
 'Bethel Nagar', 'Kirloskar Layout', 'tc.palya', 'Kodipalya',
 'Moodalapalya', 'Carmelaram', 'Hennur Bande', 'Marasandra',
 'Ullal Uppanagar', 'Jakkasandra Extension', 'Hegganahalli',
 'Dasappa Layout', 'BTM 4th Stage', 'Ullal Road',
 'Krishna Reddy Layout', 'Devi Nagar', 'Jnana Ganga Nagar',

```
'Stage-4 Bommanahalli', 'Judicial Layout, Kanakapura Road, ',
'Sathya Layout', 'Shanti Nagar', '2nd Stage Arekere Mico Layout',
'Jayamahall', 'Chennappa Layout', 'Lavakusha Nagar', 'Kadabagere',
'1st Block HBR Layout', 'Telecom Layout', 'Virupakshapura',
'Coconut Grove Layout', 'Harsha Layout',
'Ramamurthy Nagar Extension', 'Ankappa Layout', 'Pampa Extension',
'Agrahara Dasarahalli', 'Shankarapuram', 'Nagashetty Halli',
'Devara Jeevanahalli', 'RPC layout', 'Gulimangala',
'Konena Agrahara', 'Varsova Layout', '5th Block Jayanagar',
'Jayanti Nagar', 'Kammagondahalli', 'Govindapura', 'Arishinakunte',
'Ayappa Nagar'], dtype=object)
```

```
[25]: location_to_group = clusters = {
    "Electronic City & Vicinity": [
        "Electronic City", "Electronic City Phase II", "Electronics City Phase_1",
        "Bommasandra", "Bommasandra Industrial Area", "Singena Agrahara",
        "Doddathoguru",
        "Kammasandra", "Rayasandra", "Neeladri Nagar", "Stage-4 Bommanahalli",
        "Bommanahalli",
        "Hosa Road", "Huskur", "Chandapura", "Jigani", "Marsur", "Hulimavu"
    ],
    "Whitefield Cluster": [
        "Whitefield", "EPIP Zone", "ITPL", "Kadugodi", "Immadihalli",
        "Seegehalli",
        "Pattandur Agrahara", "Hagadur", "Ramagondanahalli", "Nallurhalli",
        "Hoodi", "Hoodi Circle, ",
        "Basavanapura", "Sadaramangala", "Belathur", "Kannamangala",
        "Devasthanagalu", "Varthur", "Balagere"
    ],
    "Sarjapur & Vicinity": [
        "Sarjapur", "Sarjapura - Attibele Road", "Sarjapur Road", "Sarjapur_1",
        "Kasavanhalli",
        "Kaikondrahalli", "Dommasandra", "Kodathi", "Ambalipura", "Gunjur",
        "Off Sarjapur Road, ",
        "Chikka Tirupathi"
    ],
    "JP Nagar & Surroundings": [
        "JP Nagar", "1st Phase JP Nagar", "2nd Phase JP Nagar", "3rd Phase JP_1",
        "4th Phase JP Nagar",
        "5th Phase JP Nagar", "6th Phase JP Nagar", "7th Phase JP Nagar", "8th_1",
        "9th Phase JP Nagar", "NS Palya", "Puttenahalli", "Konanakunte",
        "Talaghattapura", "Ramanjaneyanagar",
        "Sarakki Nagar", "Subramanyapura", "Vasantha Vallabha Nagar",
        "Konanakunte Cross"
```

```

],
  "Jayanagar & Basavangudi": [
    "Jayanagar", "1st Block Jayanagar", "3rd Block Jayanagar", "4th Block",
    ↪ "Jayanagar",
    "5th Block Jayanagar", "7th Block Jayanagar", "8th Block Jayanagar",
    ↪ "9th Block Jayanagar",
    "4th T block Jayanagar", "Basavangudi", "Wilson Garden",
    ↪ "Suddaguntepalya", "Hanumanth Nagar",
    "Siddapura", "KR Road"
  ],
  "Hebbal & Thanisandra Corridor": [
    "Hebbal", "Hebbal Kempapura", "Thanisandra", "Thanisandra",
    ↪ "Thanisandra Main Road",
    "Nagawara", "Nagawara Junction", "Hegde Nagar", "Amruthahalli",
    ↪ "Kodigehalli", "Jakkur", "Jakkuru Layout",
    "Tirumanahalli", "Kogilu", "Attur Layout", "Rachenahalli", "Bagalur",
    ↪ "Main Road", "Sahakara Nagar"
  ],
  "Yelahanka": [
    "Yelahanka", "Yelahanka New Town", "Singanayakanahalli",
    ↪ "Vidyaranya", "Kattigenahalli",
    "Rajankunte"
  ],
  "KR Puram & East": [
    "KR Puram", "Lakshminarayana Pura", "Battarahalli", "Dooravani Nagar",
    ↪ "Ramamurthy Nagar",
    "Ramamurthy Nagar Extension", "TC Palaya", "T.C PALYA", "Medahalli",
    ↪ "Budigere", "Old Madras Road"
  ],
  "HSR Layout & Bommanahalli": [
    "HSR Layout", "Sector 1 HSR Layout", "Sector 2 HSR Layout", "Sector 6",
    ↪ "HSR Layout", "Sector 7 HSR Layout",
    "Mangammanapalya", "Kudlu Gate", "Kudlu", "Basapura", "Singasandra",
    ↪ "Parappana Agrahara", "Iblur Village",
    "Green Glen Layout", "Somasundara Palya", "Stage-4 Bommanahalli"
  ],
  "Indiranagar & Domlur": [
    "Indira Nagar", "HAL 2nd Stage", "1st Stage Indira Nagar", "Indiranagar",
    ↪ "Club", "Domlur", "Domlur Layout",
    "New Thippasandra", "Kodihalli", "Rustam Bagh Layout", "CMH Road"
  ],
  "Marathahalli - Brookefield - ORR": [
    "Marathahalli", "Brookefield", "Dodda Nekkundi", "Dodda Nekkundi",
    ↪ "Extension", "AECS Layout", "Kundalahalli",
    "Kundalahalli Colony", "Munnekollal", "Doddanekkundi", "Thubarahalli",
    ↪ "Pattandur Agrahara", "Garudachar Palya"
  ]

```

```

],
  "Koramangala & Surroundings": [
    "Koramangala", "1st Block Koramangala", "4th Block Koramangala", "6th
↪block Koramangala",
    "8th block Koramangala", "Ejipura", "St Johns Road", "Jakkasandra
↪Extension", "Madiwala", "Roopena Agrahara"
  ],
  "Bannerghatta Road / Arekere / Gottigere": [
    "Bannerghatta Road", "Arekere", "Gottigere", "Devarachikkanahalli", "
↪Devarachikkanahalli",
    "Vijaya Bank Layout", "Bilekahalli", "Hulimavu", "Yelenahalli",
↪"Begur", "Vajralli", "Basapura"
  ],
  "Rajajinagar & Malleshwaram": [
    "Rajaji Nagar", "Malleshwaram", "Basaveshwara Nagar", "Yeshwanthpur",
↪"Peenya", "Mahalakshmi Layout",
    "Mathikere", "Jalahalli West", "Jalahalli", "Nandini Layout",
↪"Srirampura"
  ],
  "Kengeri & Rajarajeshwari Nagar": [
    "Kengeri", "Kengeri Satellite Town", "Raja Rajeshwari Nagar",
↪"Ramakrishnappa Layout",
    "Rajarajeshwari nagar", "Bisuvanahalli", "Kenchenahalli", "Mylasandra",
↪"Byatarayanapura"
  ],
  "Sarjapur Extension & South-East": [
    "Dommasandra", "Ambalipura", "Kaikondrahalli", "Kodathi", "Gunjur",
↪"Hadosiddapura",
    "Choodasandra", "Rayasandra", "Somasundara Palya"
  ],
  "West & North-West Bangalore": [
    "Nagarbhavi", "Chandra Layout", "Vishveshwarya Layout", "Vijayanagar",
↪"Ngef Layout",
    "Magadi Road", "Dasanapura", "Gollarapalya Hosahalli", "T Dasarahalli",
↪"Herohalli", "Laggere",
    "Sunkadakatte", "Mallathahalli", "Byagadadhenahalli"
  ],
  "Other South Bangalore": [
    "Chikkalasandra", "Padmanabhanagar", "Uttarahalli",
↪"Lingadheeranahalli", "Kalena Agrahara",
    "Vajarahalli", "Thurahalli", "Naganathapura", "Anjanapura",
↪"Kaggalipura", "Kanakapura Road"
  ],
  "East Bangalore Corridors": [
    "Kaggadasapura", "Mahadevpura", "Hoodi", "Hoodi Circle", " ", "Garudachar
↪Palya", "Panathur",

```

```

    "Nagondanahalli", "Bhoganhalli", "Balagere", "Bellandur", "Bellatur",
    ↪ "Varthur Road"
  ],
  "Northeast Cluster": [
    "Hennur Road", "Hennur", "Horamavu", "Horamavu Agara", "Kalyan nagar",
    ↪ "OMBR Layout",
    "Kasturi Nagar", "Kothanur", "Kothannur", "HRBR Layout",
    ↪ "Babusapalaya", "HBR Layout",
    "Kammanahalli", "Banaswadi", "Chikka Banaswadi"
  ],
  "Central Bangalore": [
    "Richmond Town", "Langford Town", "Shanthala Nagar", "Victoria Layout",
    ↪ "Cunningham Road",
    "Lavelle Road", "Mahatma Gandhi Road (MG Road)", "Ulsoor", "Cambridge
    ↪ Layout"
  ],
  "Basavangudi & Surroundings": [
    "Basavangudi", "Wilson Garden", "KR Road", "Hanumanth Nagar", "Gandhi
    ↪ Bazaar",
    "Siddapura", "V V Puram", "Lalbagh"
  ],
  "Outer and Peripheral Areas": [
    "Old Airport Road", "Mysore Road", "Mysore Highway", "Devanahalli",
    ↪ "Devanahalli Int. Airport",
    "Anekal", "Budigere", "Bisuvanahalli", "Nelamangala", "Hoskote",
    ↪ "Chikka Tirupathi",
    "Attibele", "Marsur", "Kanakapura", "Channasandra Layout",
    ↪ "Kadubeesanahalli"
  ],
  # Remnant/miscellaneous or ambiguous areas go in:
  "Others": [
    "others", "Manayata Tech Park", "Binny Pete", "Ganga Nagar", "Himagiri
    ↪ Meadows", "Lakshminarayana Pura",
    "Shampura", "LB Shastri Nagar", "MS Pallya", "Langford Town", "Green
    ↪ View Layout",
    "Shantiniketan Layout", "Dinnur", "Kannamangala", "Kalena Agrahara",
    ↪ "Cholanayakanahalli",
    "Kaval Byrasandra", "ISRO Layout", "Dasanapura", "Sanjay nagar",
    ↪ "Mylasandra", "Anandapura",
    "Doddadommasandra", "Ashwath Nagar", "Ashirvad Colony", "MCECHS
    ↪ layout", "Coffee Board Layout",
    "Motappa Layout", "Akshaya Vana", "Basavanna Nagar", "RMV Extension",
    ↪ "Tejaswini Nagar",
    "Kothnur Narayanapura", "Kumaraswami Layout", "Anekal", "Green Glen
    ↪ Layout", "Various small layouts etc."
  ]

```

```

}

def get_location_group(location):
    location = location.strip()
    for group, locations in location_to_group.items():
        if location in [loc.strip() for loc in locations]:
            return group
    return "Others" # Fallback group

df["location"] = df["location"].apply(get_location_group)

print(df["location"].unique())
print(df["location"].isnull().sum())

```

```

['Electronic City & Vicinity' 'Sarjapur & Vicinity'
 'Other South Bangalore' 'Northeast Cluster' 'Whitefield Cluster'
 'Outer and Peripheral Areas' 'Rajajinagar & Malleshwaram'
 'Marathahalli - Brookefield - ORR' 'Others' 'JP Nagar & Surroundings'
 'Bannerghatta Road / Arekere / Gottigere'
 'Kengeri & Rajarajeshwari Nagar' 'Hebbal & Thanisandra Corridor'
 'East Bangalore Corridors' 'HSR Layout & Bommanahalli' 'Yelahanka'
 'KR Puram & East' 'West & North-West Bangalore' 'Indiranagar & Domlur'
 'Koramangala & Surroundings' 'Central Bangalore'
 'Jayanagar & Basavangudi' 'Sarjapur Extension & South-East']
0

```

```

[26]: dummy = pd.get_dummies(df["location"])
df = pd.concat([df, dummy], axis = 1)
df.drop("location", axis = 1, inplace = True)

```

CONVERTING ALL OF THE DATA TYPES INTO INT FOR SIMILARITY

```

[27]: df["bath"] = df["bath"].astype(int)

```

```

[28]: df["balcony"] = df["balcony"].astype(int)

```

CAPPING THE BATH TO 5

```

[29]: df["bath"].value_counts()

```

```

[29]: bath
2      6981
3      3286
4      1226
1       788
5       524
6       273
7       102

```



```

8      64
9      43
10     13
12      7
11      3
13      3
16      2
27      1
14      1
40      1
15      1
18      1
Name: count, dtype: int64

```

```
[30]: df["bath"] = df["bath"].apply(lambda x: 5 if x > 5 else int(x))
```

```
[31]: df.describe()
```

```
[31]:
```

	bath	balcony	price	Bedroom	Hall \
count	13320.000000	13320.000000	13320.000000	13320.000000	13320.000000
mean	2.605631	1.557658	112.565627	2.780105	0.808108
std	1.004658	0.807639	148.971674	1.168284	0.393803
min	1.000000	0.000000	8.000000	1.000000	0.000000
25%	2.000000	1.000000	50.000000	2.000000	1.000000
50%	2.000000	2.000000	72.000000	3.000000	1.000000
75%	3.000000	2.000000	120.000000	3.000000	1.000000
max	5.000000	3.000000	3600.000000	9.000000	1.000000

	Kitchen
count	13320.000000
mean	0.809084
std	0.393038
min	0.000000
25%	1.000000
50%	1.000000
75%	1.000000
max	1.000000

there is a total_sqft inconsistency

```
[32]: def split_mean(column):
    column_updated = column.copy() # avoid modifying original directly
    for index, val in column.items():
        # Only process strings with "-"
        if isinstance(val, str) and '-' in val:
            parts = val.split('-')
            val1, val2 = float(parts[0].strip()), float(parts[1].strip())
            mean = (val1 + val2) / 2

```

```

        column_updated.at[index] = mean
    else:
        # Leave numeric values as-is
        try:
            column_updated.at[index] = float(val)
        except:
            column_updated.drop(index, axis = 0, inplace = True)
    return column_updated

```

```

[33]: df["total_sqft"] = split_mean(df["total_sqft"])
      df["total_sqft"] = df["total_sqft"].astype(float)

```

```

[34]: df = df.dropna(subset = ["total_sqft"])

```

```

[35]: df["sqft_per_bedroom"] = df["total_sqft"] / df["Bedroom"]

      df = df[df["sqft_per_bedroom"] >= 300]

      df = df.drop(columns=["sqft_per_bedroom"])

```

```

[36]: df.columns
      df["price"] = np.log1p(df["price"])

```

1.1 DATA PREP OVER

EDA

```

[37]: corr = df.corr()
      corr["price"].sort_values(ascending=False)

```

```

[37]: price          1.000000
      bath           0.721975
      Bedroom        0.655195
      total_sqft      0.581916
      Plot Area       0.344026
      balcony         0.150297
      Rajajinagar & Malleshwaram 0.138118
      Central Bangalore 0.125209
      Indiranagar & Domlur 0.112415
      Jayanagar & Basavangudi 0.094847
      Koramangala & Surroundings 0.086527
      Ready To Move    0.080647
      Others           0.053139
      Hebbal & Thanisandra Corridor 0.052582
      Marathahalli - Brookefield - ORR 0.021725
      Mar              0.015543
      Sarjapur & Vicinity 0.014344
      Feb              0.014299
      Whitefield Cluster 0.013550

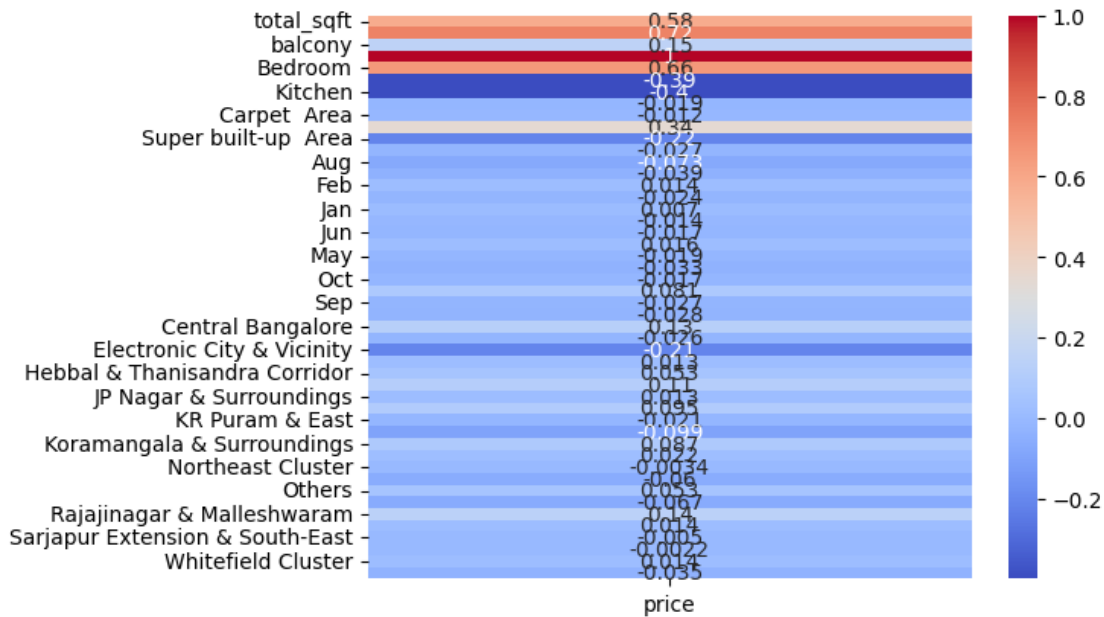
```

JP Nagar & Surroundings	0.013481
HSR Layout & Bommanahalli	0.012554
Jan	0.006986
West & North-West Bangalore	-0.002157
Northeast Cluster	-0.003385
Sarjapur Extension & South-East	-0.004994
Carpet Area	-0.011509
Jul	-0.013894
Oct	-0.016503
Jun	-0.016664
May	-0.018623
Built-up Area	-0.019281
KR Puram & East	-0.021436
Immediate Possession	-0.023715
East Bangalore Corridors	-0.025508
Sep	-0.026566
Apr	-0.026675
Bannerghatta Road / Arekere / Gottigere	-0.028466
Nov	-0.033058
Yelahanka	-0.034774
Dec	-0.038741
Other South Bangalore	-0.060443
Outer and Peripheral Areas	-0.067456
Aug	-0.072811
Kengeri & Rajarajeshwari Nagar	-0.098803
Electronic City & Vicinity	-0.211997
Super built-up Area	-0.216936
Hall	-0.393623
Kitchen	-0.398680

Name: price, dtype: float64

```
[38]: sns.heatmap(df.corr()[["price"]], annot=True, cmap="coolwarm")
```

```
[38]: <Axes: >
```



```
[39]: corr = df.corr()
price_log_corr = corr["price"].sort_values(ascending = False)
colors = ['green' if val > 0 else 'red' for val in price_log_corr]

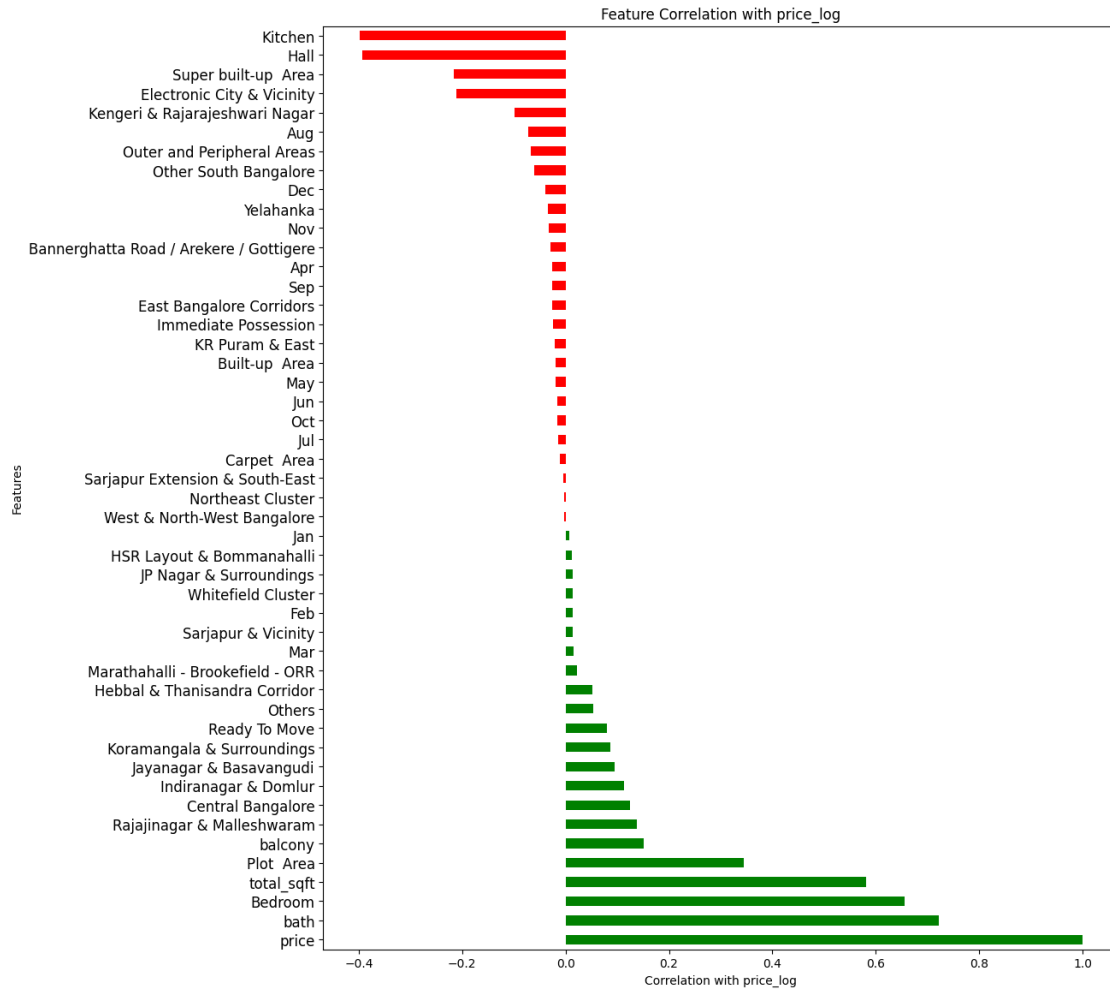
# Create the figure with desired size BEFORE plotting
plt.figure(figsize=(12, 14))

# Plot horizontal bar chart
price_log_corr.plot(kind="barh", color=colors)

# Set labels and appearance AFTER plotting
plt.xlabel("Correlation with price_log")
plt.ylabel("Features")
plt.yticks(fontsize=12) # Adjust fontsize for readability
plt.yticks(rotation=0) # Horizontal labels for y-axis look better

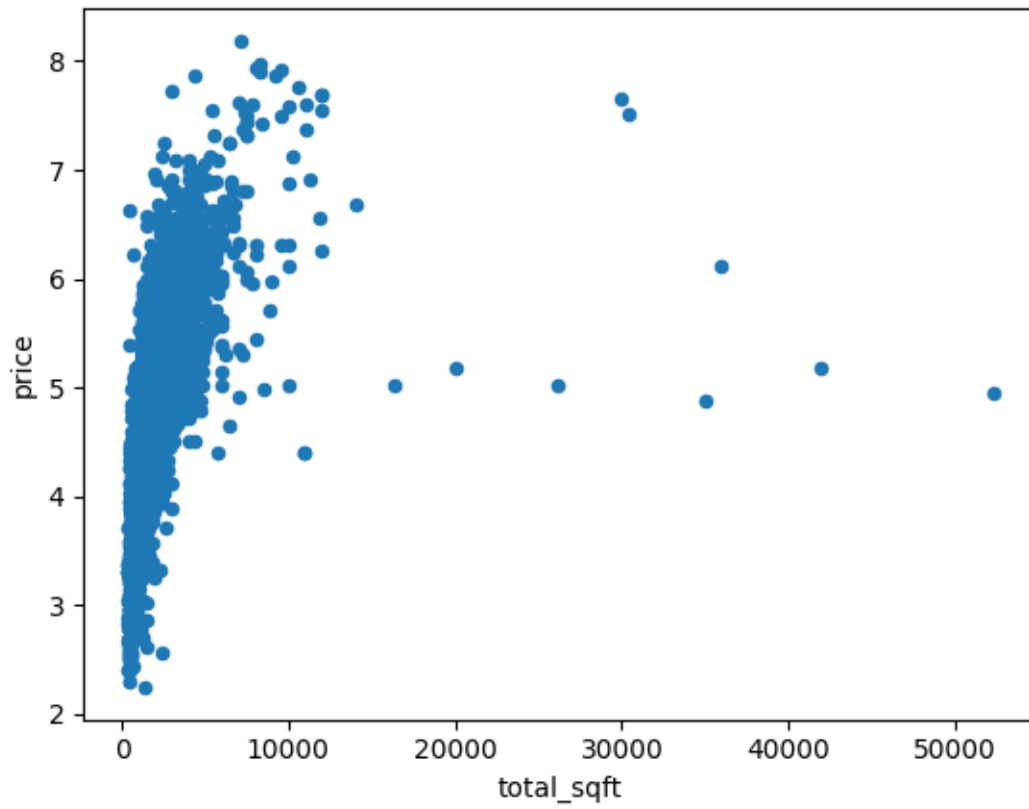
plt.title("Feature Correlation with price_log")

plt.show()
```



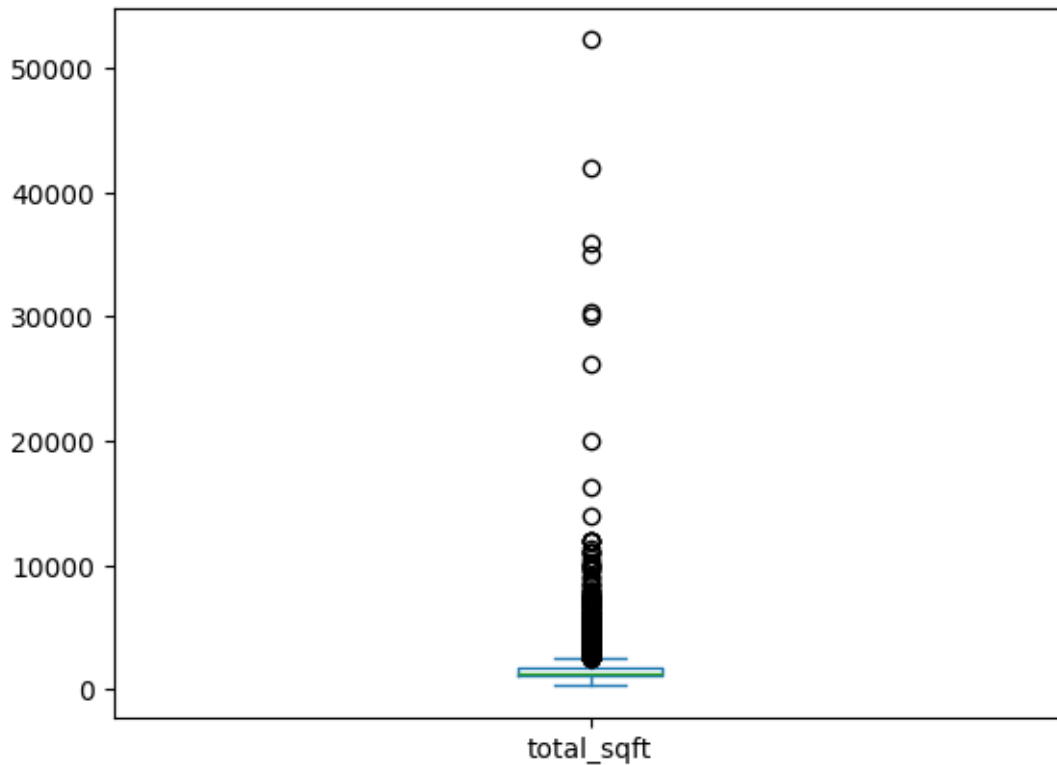
```
[40]: df.plot(kind = "scatter", x = "total_sqft", y= "price")
```

```
[40]: <Axes: xlabel='total_sqft', ylabel='price'>
```



```
[41]: df["total_sqft"].plot(kind = "box")
```

```
[41]: <Axes: >
```



MODELLING

```
[42]: x_data = df.drop("price", axis=1)
      y_data = df["price"]
```

```
[43]: x_train, x_test, y_train, y_test = train_test_split(x_data, y_data, test_size = 0.3, random_state = 0)
```

```
[44]: z = LinearRegression()
      z.fit(x_train, y_train)
```

```
[44]: LinearRegression()
```

```
[45]: y_predicted = z.predict(x_test)
      print(f"equation Y = {z.coef_}*X + {z.intercept_}")
```

```
equation Y = [ 1.28550344e-04  3.14390685e-01  1.69789077e-02  9.41159387e-02
 3.64187531e-01 -5.06566352e-01 -7.87171113e-02 -5.22662454e-02
 1.62709245e-01 -3.17258885e-02  3.14737969e-02 -1.12220375e-01
 6.22021625e-03  9.09423116e-02 -1.57130948e-02  1.14863998e-01
 1.42446787e-02 -3.14497475e-02  1.58704014e-01  9.39042840e-04
-1.27969803e-01 -8.35640469e-02  2.46443837e-02 -7.11153758e-02
-1.84663238e-01  7.37625060e-01 -1.04674008e-01 -3.63157649e-01
-3.25577705e-02  1.57339838e-02  5.16301648e-01 -2.53024319e-03]
```

```

5.36315984e-01 -1.78965502e-01 -3.16353573e-01 3.97617112e-01
-4.27199716e-02 -1.02965922e-01 -2.03477483e-01 -8.23121584e-02
-3.54978431e-01 4.00406512e-01 -1.08540768e-01 -1.84217137e-01
-6.53500005e-02 -8.52903394e-02 -1.91246104e-01]*X + 3.327860315862246

```

```
[46]: z.score(x_test, y_test)
```

```
[46]: 0.6747972540273022
```

```
[47]: mse = mean_squared_error(y_test, y_predicted)
mae = mean_absolute_error(y_test, y_predicted)
print(f"mae is {mae}\nmse is {mse}")
```

```

mae is 0.2984970094351328
mse is 0.16254232359351314

```

```
[48]: metrics = pd.DataFrame({
    'linear': {
        'mae': 0.28021235885722107,
        'mse': 0.134471296817324,
        'r2': 0.7285537598905569
    }
})

print(metrics)
```

```

      linear
mae  0.280212
mse  0.134471
r2   0.728554

```

```
[49]: ax1 = sns.distplot(y_test, hist = False, color = "r", label = "Actual Value")
sns.distplot(y_predicted, hist = False, color = "b", label = "Predicted", ax = ax1)
plt.legend()
```

C:\Users\shrih\AppData\Local\Temp\ipykernel_8584\1967961399.py:1: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `kdeplot` (an axes-level function for kernel density plots).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
ax1 = sns.distplot(y_test, hist = False, color = "r", label = "Actual Value")
```

C:\Users\shrih\AppData\Local\Temp\ipykernel_8584\1967961399.py:2: UserWarning:

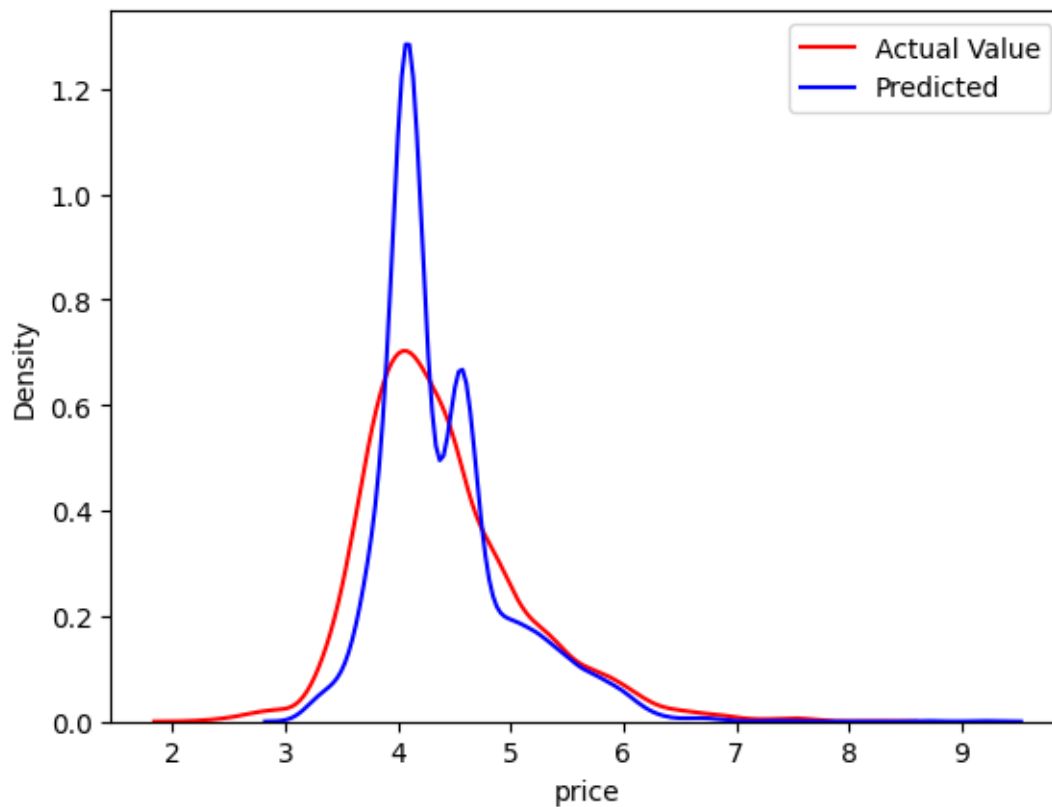
`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `kdeplot` (an axes-level function for kernel density plots).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
sns.distplot(y_predicted, hist = False, color = "b", label = "Predicted", ax = ax1)
```

[49]: <matplotlib.legend.Legend at 0x26b3bd5b8c0>



```
[50]: print("something")

poly = PolynomialFeatures(degree=2)
print(f"Running degree2")
x_train_poly = poly.fit_transform(x_train)
x_test_poly = poly.transform(x_test)
```

```

model = LinearRegression()
model.fit(x_train_poly, y_train)

y_pred = model.predict(x_test_poly)

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
r2 = model.score(x_test_poly, y_test)

print(f"Degree 2 -> mae: {mae}, mse: {mse}, r2: {r2}")

metrics.loc[["mae", "mse", "r2"], "degree2"] = [mae, mse, r2]

```

```

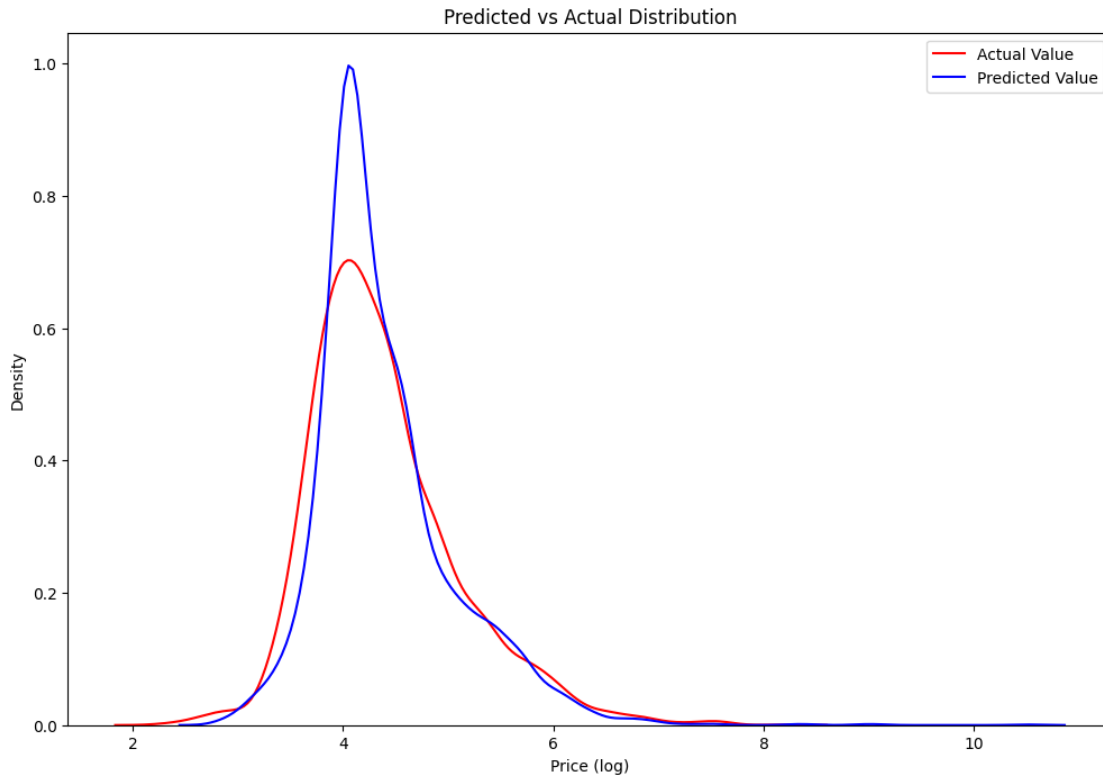
something
Running degree2
Degree 2 -> mae: 0.25475037269149753, mse: 0.136015906265097, r2:
0.7278693620500213

```

```

[51]: plt.figure(figsize=(12,8))
sns.kdeplot(y_test, color="r", label="Actual Value")
sns.kdeplot(y_pred, color="b", label="Predicted Value")
plt.xlabel("Price (log)")
plt.ylabel("Density")
plt.title("Predicted vs Actual Distribution")
plt.legend()
plt.show()

```



```
[52]: print("hello")
```

hello

```
[53]: RidgeModel = Ridge(alpha = 0.1)
RidgeModel.fit(x_train, y_train)
y_predicted = RidgeModel.predict(x_test)
```

```
[54]: RidgeModel.score(x_test, y_test)
```

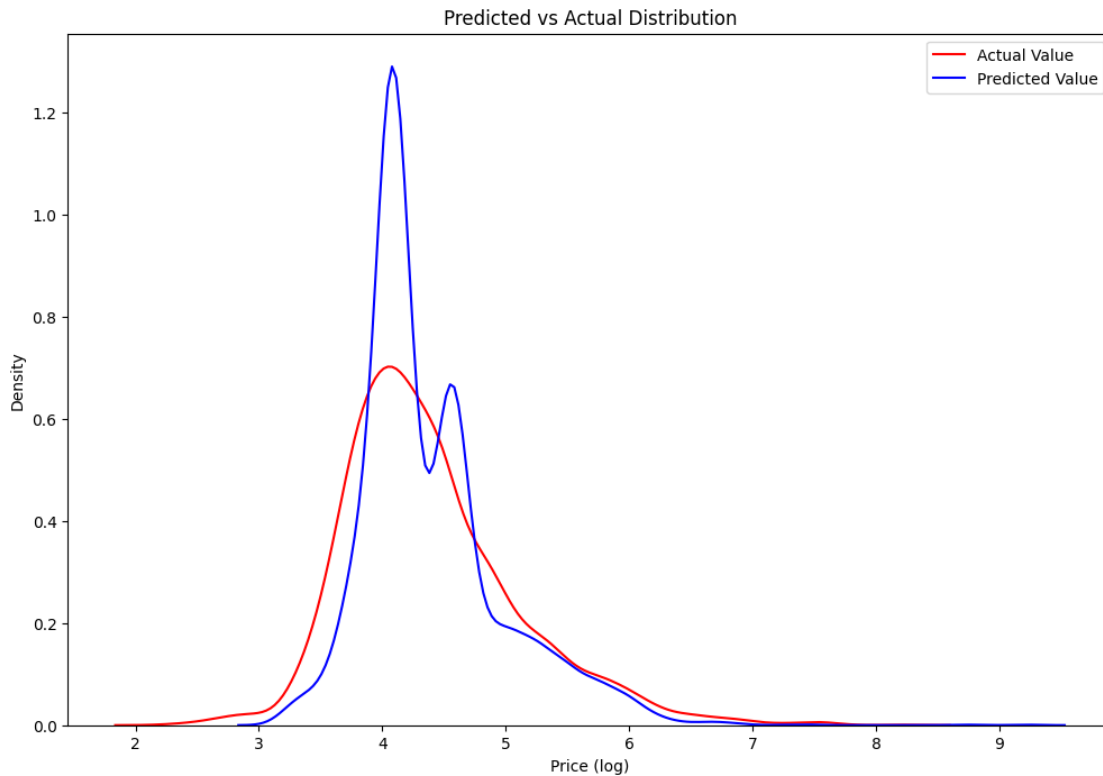
```
[54]: 0.6748302236807104
```

```
[55]: r2 = r2_score(y_test, y_predicted)
mae = mean_absolute_error(y_test, y_predicted)
mse = mean_squared_error(y_test, y_predicted)
```

```
[56]: metrics.loc[["mae", "mse", "r2"], "Ridge(0.1)"] = [mae, mse, r2]
```

```
[57]: plt.figure(figsize=(12,8))
sns.kdeplot(y_test, color="r", label="Actual Value")
sns.kdeplot(y_predicted, color="b", label="Predicted Value")
plt.xlabel("Price (log)")
plt.ylabel("Density")
```

```
plt.title("Predicted vs Actual Distribution")
plt.legend()
plt.show()
```



```
[58]: parameters = [{"alpha" : [0.001, 0.01, 0.1, 1, 10, 25, 50, 75, 100, 250, 500,
    ↪750, 1000]}]
RidgeModel = Ridge(max_iter = 1000)
Grid1 = GridSearchCV(estimator = RidgeModel, param_grid = parameters, cv = 5,
    ↪scoring = "r2", verbose = 2)
Grid1.fit(x_train, y_train)
```

Fitting 5 folds for each of 13 candidates, totalling 65 fits

```
[CV] END ...alpha=0.001; total time= 0.0s
[CV] END ...alpha=0.001; total time= 0.0s
[CV] END ...alpha=0.001; total time= 0.0s
[CV] END ...alpha=0.001; total time= 0.0s
[CV] END ...alpha=0.001; total time= 0.0s
[CV] END ...alpha=0.01; total time= 0.0s
[CV] END ...alpha=0.01; total time= 0.0s
[CV] END ...alpha=0.01; total time= 0.0s
[CV] END ...alpha=0.01; total time= 0.0s
[CV] END ...alpha=0.01; total time= 0.0s
```

[illegible]

```
[CV] END ...alpha=750; total time= 0.0s
[CV] END ...alpha=750; total time= 0.0s
[CV] END ...alpha=1000; total time= 0.0s
[CV] END ...alpha=1000; total time= 0.0s
[CV] END ...alpha=1000; total time= 0.0s
[CV] END ...alpha=1000; total time= 0.0s
[CV] END ...alpha=1000; total time= 0.0s
```

```
[58]: GridSearchCV(cv=5, estimator=Ridge(max_iter=1000),
                param_grid=[{'alpha': [0.001, 0.01, 0.1, 1, 10, 25, 50, 75, 100,
                                         250, 500, 750, 1000]}],
                scoring='r2', verbose=2)
```

```
[59]: print(Grid1.best_estimator_)
      print(Grid1.best_params_)
      r2 = Grid1.score(x_test, y_test)
      print(r2)
```

```
Ridge(alpha=1, max_iter=1000)
{'alpha': 1}
0.6750252368552159
```

```
[60]: y_pred = Grid1.predict(x_test)

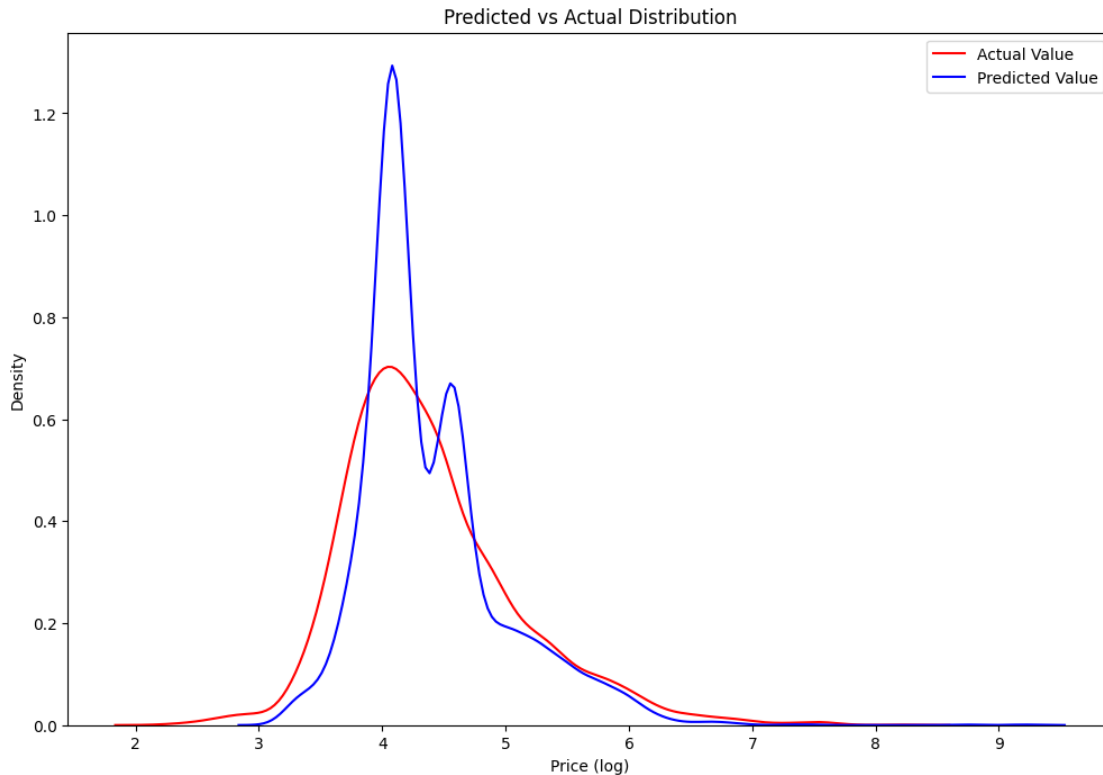
mae = mean_absolute_error(y_test, y_pred)

mse = mean_squared_error(y_test, y_pred)

print(f"MAE: {mae}")
print(f"MSE: {mse}")
metrics.loc[["mae", "mse", "r2"], "Ridge(1)"] = [mae, mse, r2]
```

```
MAE: 0.29838832903698836
MSE: 0.16242837357603201
```

```
[61]: plt.figure(figsize=(12,8))
      sns.kdeplot(y_test, color="r", label="Actual Value")
      sns.kdeplot(y_pred, color="b", label="Predicted Value")
      plt.xlabel("Price (log)")
      plt.ylabel("Density")
      plt.title("Predicted vs Actual Distribution")
      plt.legend()
      plt.show()
```



```
[62]: lasso = Lasso(max_iter = 1000)
Grid2 = GridSearchCV(estimator = lasso, param_grid = parameters, scoring = "r2", verbose = 2)
Grid2.fit(x_train, y_train)
y_predicted = Grid2.predict(x_test)

print(Grid2.best_estimator_)
alpha = Grid2.best_params_["alpha"]
r2 = Grid2.score(x_test, y_test)
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
print(f"MAE: {mae}")
print(f"MSE: {mse}")
print(f"R2: {r2}")
metrics.loc[["mae", "mse", "r2"], f"Lasso{alpha}"] = [mae, mse, r2]
sns.kdeplot(y_test, color = "r", label = "Actual Value")
sns.kdeplot(y_predicted, color = "b", label = "Predicted Value")
```

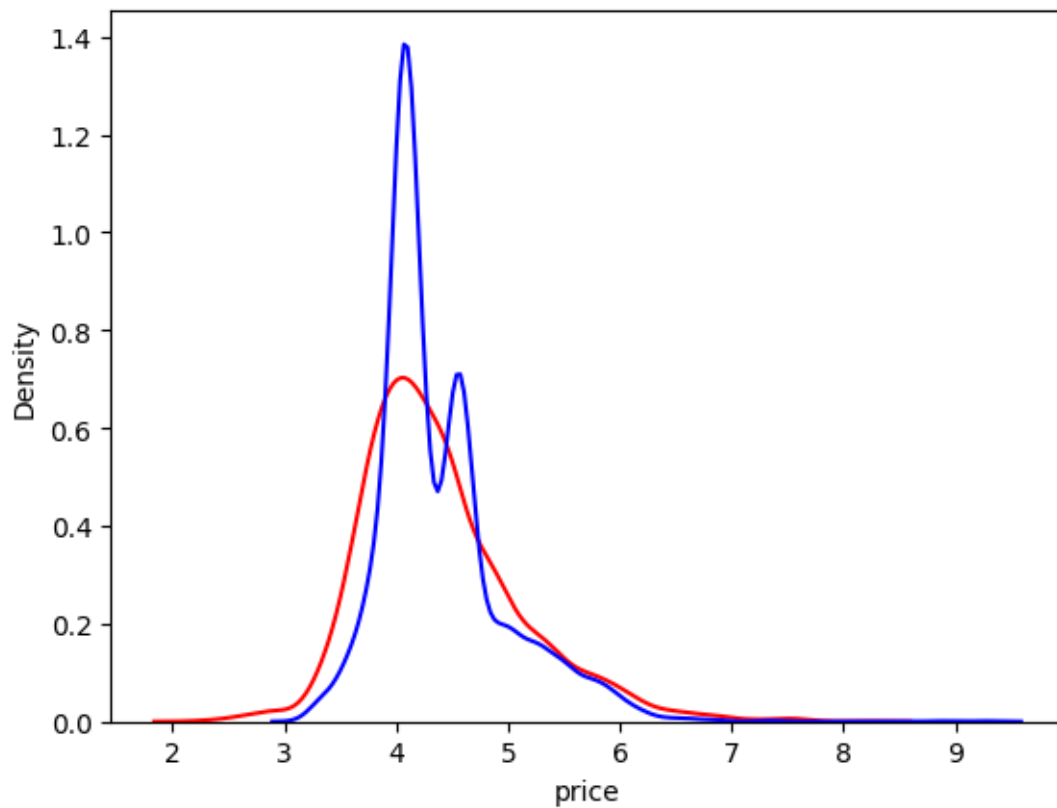
Fitting 5 folds for each of 13 candidates, totalling 65 fits

```
[CV] END ...alpha=0.001; total time= 0.0s
[CV] END ...alpha=0.001; total time= 0.0s
[CV] END ...alpha=0.001; total time= 0.0s
[CV] END ...alpha=0.001; total time= 0.0s
```

[illegible]


```
[CV] END ...alpha=500; total time= 0.0s
[CV] END ...alpha=500; total time= 0.0s
[CV] END ...alpha=500; total time= 0.0s
[CV] END ...alpha=750; total time= 0.0s
[CV] END ...alpha=750; total time= 0.0s
[CV] END ...alpha=750; total time= 0.0s
[CV] END ...alpha=750; total time= 0.0s
[CV] END ...alpha=750; total time= 0.0s
[CV] END ...alpha=1000; total time= 0.0s
[CV] END ...alpha=1000; total time= 0.0s
[CV] END ...alpha=1000; total time= 0.0s
[CV] END ...alpha=1000; total time= 0.0s
[CV] END ...alpha=1000; total time= 0.0s
Lasso(alpha=0.001)
MAE: 0.29838832903698836
MSE: 0.16242837357603201
R2; 0.6737846842891375
```

```
[62]: <Axes: xlabel='price', ylabel='Density'>
```



```
[63]: #metrics.rename(columns={"Lasso(alpha)": "Lasso(0.01)"}, inplace=True)
parameters = {
    "max_depth": [3, 5, 6, 7, 8, 9, 10, 15],
    "min_samples_split": [2, 5, 8, 9, 10, 11, 12, 15],
    "min_samples_leaf": [1, 2, 5]
}

# Initialize the Decision Tree Regressor
dtree = DecisionTreeRegressor(random_state=42)

Grid3 = GridSearchCV(estimator=dtree, param_grid=parameters,
                     scoring="r2", cv=6, verbose=2)

Grid3.fit(x_train, y_train)

# Predict on test data
y_pred_tree = Grid3.predict(x_test)

print("Best Decision Tree:", Grid3.best_estimator_)
best_params_tree = Grid3.best_params_

r2 = Grid3.score(x_test, y_test)
mae = mean_absolute_error(y_test, y_pred_tree)
mse = mean_squared_error(y_test, y_pred_tree)
```

Fitting 6 folds for each of 192 candidates, totalling 1152 fits

```
[CV] END max_depth=3, min_samples_leaf=1, min_samples_split=2; total time=
0.0s
[CV] END max_depth=3, min_samples_leaf=1, min_samples_split=2; total time=
0.0s
[CV] END max_depth=3, min_samples_leaf=1, min_samples_split=2; total time=
0.0s
[CV] END max_depth=3, min_samples_leaf=1, min_samples_split=2; total time=
0.0s
[CV] END max_depth=3, min_samples_leaf=1, min_samples_split=2; total time=
0.0s
[CV] END max_depth=3, min_samples_leaf=1, min_samples_split=2; total time=
0.0s
[CV] END max_depth=3, min_samples_leaf=1, min_samples_split=2; total time=
0.0s
[CV] END max_depth=3, min_samples_leaf=1, min_samples_split=5; total time=
0.0s
[CV] END max_depth=3, min_samples_leaf=1, min_samples_split=5; total time=
0.0s
[CV] END max_depth=3, min_samples_leaf=1, min_samples_split=5; total time=
0.0s
[CV] END max_depth=3, min_samples_leaf=1, min_samples_split=5; total time=
0.0s
[CV] END max_depth=3, min_samples_leaf=1, min_samples_split=5; total time=
0.0s
```


[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]


```

[CV] END max_depth=15, min_samples_leaf=5, min_samples_split=11; total time=
0.0s
[CV] END max_depth=15, min_samples_leaf=5, min_samples_split=12; total time=
0.0s
[CV] END max_depth=15, min_samples_leaf=5, min_samples_split=12; total time=
0.0s
[CV] END max_depth=15, min_samples_leaf=5, min_samples_split=12; total time=
0.0s
[CV] END max_depth=15, min_samples_leaf=5, min_samples_split=12; total time=
0.0s
[CV] END max_depth=15, min_samples_leaf=5, min_samples_split=12; total time=
0.0s
[CV] END max_depth=15, min_samples_leaf=5, min_samples_split=12; total time=
0.0s
[CV] END max_depth=15, min_samples_leaf=5, min_samples_split=12; total time=
0.0s
[CV] END max_depth=15, min_samples_leaf=5, min_samples_split=15; total time=
0.0s
[CV] END max_depth=15, min_samples_leaf=5, min_samples_split=15; total time=
0.0s
[CV] END max_depth=15, min_samples_leaf=5, min_samples_split=15; total time=
0.0s
[CV] END max_depth=15, min_samples_leaf=5, min_samples_split=15; total time=
0.0s
[CV] END max_depth=15, min_samples_leaf=5, min_samples_split=15; total time=
0.0s
[CV] END max_depth=15, min_samples_leaf=5, min_samples_split=15; total time=
0.0s
Best Decision Tree: DecisionTreeRegressor(max_depth=10, min_samples_leaf=5,
min_samples_split=15,
                                random_state=42)

```

```

[64]: print(f"MAE: {mae}")
      print(f"MSE: {mse}")
      print(f"R2: {r2}")

      print(Grid3.best_estimator_)
      print(Grid3.best_params_)
      # Store metrics in your metrics DataFrame
      metrics.loc[["mae", "mse", "r2"],_]
          ↳ f"DecisionTreeRegressor{best_params_tree['max_depth']}" = [mae, mse, r2]

      # Plot predicted vs actual
      plt.figure(figsize=(8,6))
      sns.kdeplot(y_test, color="r", label="Actual Value")
      sns.kdeplot(y_pred_tree, color="b", label="Predicted Value")
      plt.title("Decision Tree Regression: Actual vs Predicted")
      plt.legend()
      plt.show()

```

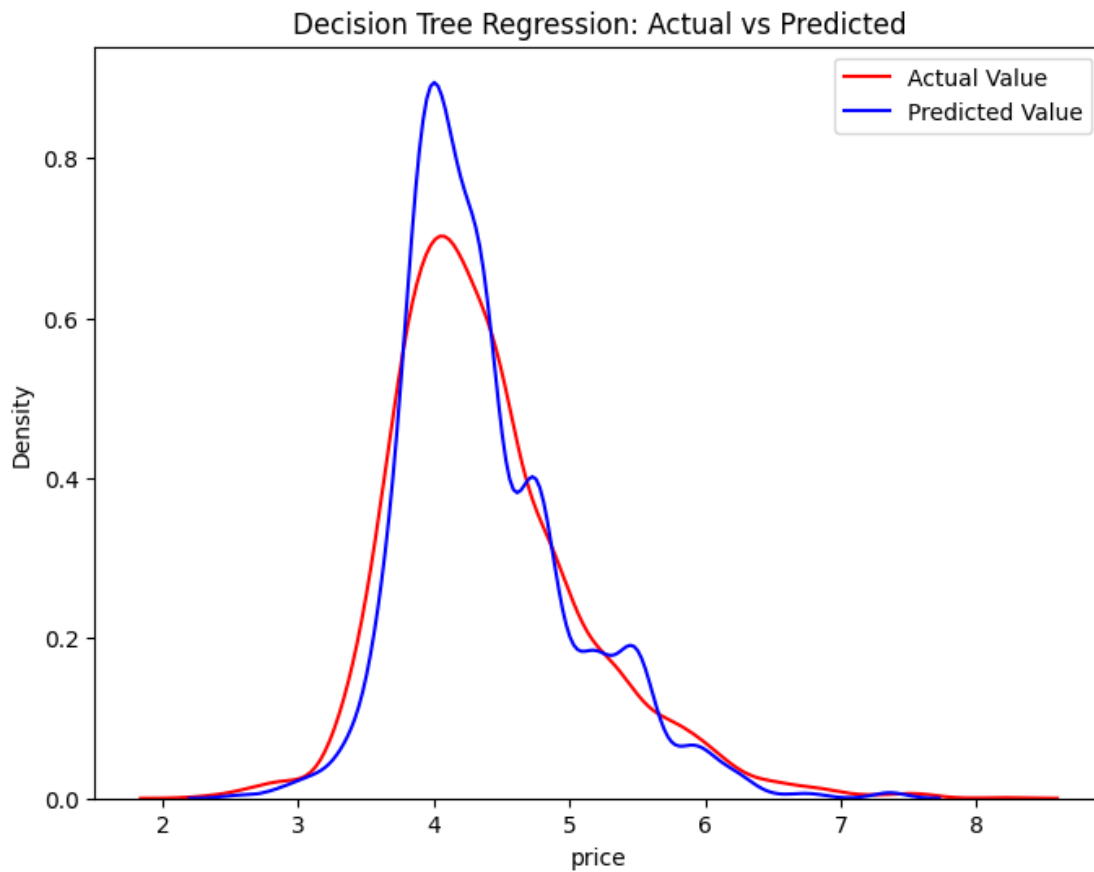
MAE: 0.2532539356529356

MSE: 0.12249935360055325

R^2 : 0.7549122881348409

DecisionTreeRegressor(max_depth=10, min_samples_leaf=5, min_samples_split=15,
random_state=42)

{'max_depth': 10, 'min_samples_leaf': 5, 'min_samples_split': 15}



```
[65]: parameters_xgb = {  
    "n_estimators": [100, 200],  
    "max_depth": [3, 5, 7],  
    "learning_rate": [0.01, 0.1, 0.2],  
    "subsample": [0.7, 1.0]  
}  
  
# Initialize the XGBoost Regressor  
xgb = XGBRegressor(random_state=42, objective='reg:squarederror')  
  
# GridSearchCV  
Grid4 = GridSearchCV(estimator=xgb, param_grid=parameters_xgb,
```

```

        scoring="r2", cv=5, verbose=2)

# Fit on training data
Grid4.fit(x_train, y_train) # ensure y is 1D

# Predict on test data
y_pred_xgb = Grid4.predict(x_test)

# Best estimator and parameters
print("Best XGBoost model:", Grid4.best_estimator_)
best_params_xgb = Grid4.best_params_
print(best_params_xgb)
print(Grid4.best_estimator_)

# Metrics
r2 = Grid4.score(x_test, y_test)
mae = mean_absolute_error(y_test, y_pred_xgb)
mse = mean_squared_error(y_test, y_pred_xgb)

```

Fitting 5 folds for each of 36 candidates, totalling 180 fits

```

[CV] END learning_rate=0.01, max_depth=3, n_estimators=100, subsample=0.7; total
time= 2.0s
[CV] END learning_rate=0.01, max_depth=3, n_estimators=100, subsample=0.7; total
time= 0.4s
[CV] END learning_rate=0.01, max_depth=3, n_estimators=100, subsample=0.7; total
time= 0.2s
[CV] END learning_rate=0.01, max_depth=3, n_estimators=100, subsample=0.7; total
time= 0.5s
[CV] END learning_rate=0.01, max_depth=3, n_estimators=100, subsample=0.7; total
time= 0.5s
[CV] END learning_rate=0.01, max_depth=3, n_estimators=100, subsample=1.0; total
time= 0.4s
[CV] END learning_rate=0.01, max_depth=3, n_estimators=100, subsample=1.0; total
time= 0.3s
[CV] END learning_rate=0.01, max_depth=3, n_estimators=100, subsample=1.0; total
time= 0.3s
[CV] END learning_rate=0.01, max_depth=3, n_estimators=100, subsample=1.0; total
time= 0.3s
[CV] END learning_rate=0.01, max_depth=3, n_estimators=100, subsample=1.0; total
time= 0.2s
[CV] END learning_rate=0.01, max_depth=3, n_estimators=200, subsample=0.7; total
time= 0.7s
[CV] END learning_rate=0.01, max_depth=3, n_estimators=200, subsample=0.7; total
time= 0.7s
[CV] END learning_rate=0.01, max_depth=3, n_estimators=200, subsample=0.7; total
time= 0.7s
[CV] END learning_rate=0.01, max_depth=3, n_estimators=200, subsample=0.7; total
time= 0.7s

```


[CV] END learning_rate=0.01, max_depth=5, n_estimators=200, subsample=1.0; total time= 0.9s

[CV] END learning_rate=0.01, max_depth=5, n_estimators=200, subsample=1.0; total time= 0.9s

[CV] END learning_rate=0.01, max_depth=7, n_estimators=100, subsample=0.7; total time= 0.8s

[CV] END learning_rate=0.01, max_depth=7, n_estimators=100, subsample=0.7; total time= 0.8s

[CV] END learning_rate=0.01, max_depth=7, n_estimators=100, subsample=0.7; total time= 0.7s

[CV] END learning_rate=0.01, max_depth=7, n_estimators=100, subsample=0.7; total time= 0.9s

[CV] END learning_rate=0.01, max_depth=7, n_estimators=100, subsample=0.7; total time= 0.7s

[CV] END learning_rate=0.01, max_depth=7, n_estimators=100, subsample=1.0; total time= 0.7s

[CV] END learning_rate=0.01, max_depth=7, n_estimators=100, subsample=1.0; total time= 0.7s

[CV] END learning_rate=0.01, max_depth=7, n_estimators=100, subsample=1.0; total time= 0.8s

[CV] END learning_rate=0.01, max_depth=7, n_estimators=100, subsample=1.0; total time= 0.7s

[CV] END learning_rate=0.01, max_depth=7, n_estimators=100, subsample=1.0; total time= 0.7s

[CV] END learning_rate=0.01, max_depth=7, n_estimators=100, subsample=1.0; total time= 0.7s

[CV] END learning_rate=0.01, max_depth=7, n_estimators=200, subsample=0.7; total time= 1.5s

[CV] END learning_rate=0.01, max_depth=7, n_estimators=200, subsample=0.7; total time= 1.6s

[CV] END learning_rate=0.01, max_depth=7, n_estimators=200, subsample=0.7; total time= 1.6s

[CV] END learning_rate=0.01, max_depth=7, n_estimators=200, subsample=0.7; total time= 1.6s

[CV] END learning_rate=0.01, max_depth=7, n_estimators=200, subsample=0.7; total time= 1.7s

[CV] END learning_rate=0.01, max_depth=7, n_estimators=200, subsample=1.0; total time= 1.7s

[CV] END learning_rate=0.01, max_depth=7, n_estimators=200, subsample=1.0; total time= 1.5s

[CV] END learning_rate=0.01, max_depth=7, n_estimators=200, subsample=1.0; total time= 1.5s

[CV] END learning_rate=0.01, max_depth=7, n_estimators=200, subsample=1.0; total time= 1.6s

[CV] END learning_rate=0.01, max_depth=7, n_estimators=200, subsample=1.0; total time= 2.8s

[CV] END learning_rate=0.1, max_depth=3, n_estimators=100, subsample=0.7; total time= 0.3s

[CV] END learning_rate=0.1, max_depth=3, n_estimators=100, subsample=0.7; total time= 0.4s

[illegible]

[illegible]

[CV] END learning_rate=0.1, max_depth=7, n_estimators=200, subsample=0.7; total time= 1.5s

[CV] END learning_rate=0.1, max_depth=7, n_estimators=200, subsample=0.7; total time= 1.6s

[CV] END learning_rate=0.1, max_depth=7, n_estimators=200, subsample=0.7; total time= 1.6s

[CV] END learning_rate=0.1, max_depth=7, n_estimators=200, subsample=0.7; total time= 1.5s

[CV] END learning_rate=0.1, max_depth=7, n_estimators=200, subsample=0.7; total time= 1.6s

[CV] END learning_rate=0.1, max_depth=7, n_estimators=200, subsample=1.0; total time= 1.4s

[CV] END learning_rate=0.1, max_depth=7, n_estimators=200, subsample=1.0; total time= 1.3s

[CV] END learning_rate=0.1, max_depth=7, n_estimators=200, subsample=1.0; total time= 1.2s

[CV] END learning_rate=0.1, max_depth=7, n_estimators=200, subsample=1.0; total time= 1.4s

[CV] END learning_rate=0.1, max_depth=7, n_estimators=200, subsample=1.0; total time= 1.5s

[CV] END learning_rate=0.2, max_depth=3, n_estimators=100, subsample=0.7; total time= 0.4s

[CV] END learning_rate=0.2, max_depth=3, n_estimators=100, subsample=0.7; total time= 0.4s

[CV] END learning_rate=0.2, max_depth=3, n_estimators=100, subsample=0.7; total time= 0.3s

[CV] END learning_rate=0.2, max_depth=3, n_estimators=100, subsample=0.7; total time= 0.4s

[CV] END learning_rate=0.2, max_depth=3, n_estimators=100, subsample=0.7; total time= 0.3s

[CV] END learning_rate=0.2, max_depth=3, n_estimators=100, subsample=1.0; total time= 0.2s

[CV] END learning_rate=0.2, max_depth=3, n_estimators=100, subsample=1.0; total time= 0.4s

[CV] END learning_rate=0.2, max_depth=3, n_estimators=100, subsample=1.0; total time= 0.4s

[CV] END learning_rate=0.2, max_depth=3, n_estimators=100, subsample=1.0; total time= 0.3s

[CV] END learning_rate=0.2, max_depth=3, n_estimators=100, subsample=1.0; total time= 0.3s

[CV] END learning_rate=0.2, max_depth=3, n_estimators=200, subsample=0.7; total time= 0.7s

[CV] END learning_rate=0.2, max_depth=3, n_estimators=200, subsample=0.7; total time= 0.9s

[CV] END learning_rate=0.2, max_depth=3, n_estimators=200, subsample=0.7; total time= 0.7s

[CV] END learning_rate=0.2, max_depth=3, n_estimators=200, subsample=0.7; total time= 0.7s

[CV] END learning_rate=0.2, max_depth=5, n_estimators=200, subsample=1.0; total time= 0.9s

[CV] END learning_rate=0.2, max_depth=5, n_estimators=200, subsample=1.0; total time= 0.9s

[CV] END learning_rate=0.2, max_depth=7, n_estimators=100, subsample=0.7; total time= 0.7s

[CV] END learning_rate=0.2, max_depth=7, n_estimators=100, subsample=0.7; total time= 0.8s

[CV] END learning_rate=0.2, max_depth=7, n_estimators=100, subsample=0.7; total time= 0.7s

[CV] END learning_rate=0.2, max_depth=7, n_estimators=100, subsample=0.7; total time= 0.7s

[CV] END learning_rate=0.2, max_depth=7, n_estimators=100, subsample=0.7; total time= 0.7s

[CV] END learning_rate=0.2, max_depth=7, n_estimators=100, subsample=1.0; total time= 0.7s

[CV] END learning_rate=0.2, max_depth=7, n_estimators=100, subsample=1.0; total time= 0.6s

[CV] END learning_rate=0.2, max_depth=7, n_estimators=100, subsample=1.0; total time= 0.7s

[CV] END learning_rate=0.2, max_depth=7, n_estimators=100, subsample=1.0; total time= 0.6s

[CV] END learning_rate=0.2, max_depth=7, n_estimators=100, subsample=1.0; total time= 0.6s

[CV] END learning_rate=0.2, max_depth=7, n_estimators=200, subsample=0.7; total time= 1.5s

[CV] END learning_rate=0.2, max_depth=7, n_estimators=200, subsample=0.7; total time= 1.4s

[CV] END learning_rate=0.2, max_depth=7, n_estimators=200, subsample=0.7; total time= 1.5s

[CV] END learning_rate=0.2, max_depth=7, n_estimators=200, subsample=0.7; total time= 1.5s

[CV] END learning_rate=0.2, max_depth=7, n_estimators=200, subsample=0.7; total time= 1.4s

[CV] END learning_rate=0.2, max_depth=7, n_estimators=200, subsample=1.0; total time= 1.4s

[CV] END learning_rate=0.2, max_depth=7, n_estimators=200, subsample=1.0; total time= 1.3s

[CV] END learning_rate=0.2, max_depth=7, n_estimators=200, subsample=1.0; total time= 1.3s

[CV] END learning_rate=0.2, max_depth=7, n_estimators=200, subsample=1.0; total time= 1.4s

[CV] END learning_rate=0.2, max_depth=7, n_estimators=200, subsample=1.0; total time= 1.3s

Best XGBoost model: XGBRegressor(base_score=None, booster=None, callbacks=None, colsample_bylevel=None, colsample_bynode=None, colsample_bytree=None, device=None, early_stopping_rounds=None, enable_categorical=False, eval_metric=None, feature_types=None,

```

feature_weights=None, gamma=None, grow_policy=None,
importance_type=None, interaction_constraints=None,
learning_rate=0.1, max_bin=None, max_cat_threshold=None,
max_cat_to_onehot=None, max_delta_step=None, max_depth=7,
max_leaves=None, min_child_weight=None, missing=nan,
monotone_constraints=None, multi_strategy=None, n_estimators=100,
n_jobs=None, num_parallel_tree=None, ...)
{'learning_rate': 0.1, 'max_depth': 7, 'n_estimators': 100, 'subsample': 1.0}
XGBRegressor(base_score=None, booster=None, callbacks=None,
colsample_bylevel=None, colsample_bynode=None,
colsample_bytree=None, device=None, early_stopping_rounds=None,
enable_categorical=False, eval_metric=None, feature_types=None,
feature_weights=None, gamma=None, grow_policy=None,
importance_type=None, interaction_constraints=None,
learning_rate=0.1, max_bin=None, max_cat_threshold=None,
max_cat_to_onehot=None, max_delta_step=None, max_depth=7,
max_leaves=None, min_child_weight=None, missing=nan,
monotone_constraints=None, multi_strategy=None, n_estimators=100,
n_jobs=None, num_parallel_tree=None, ...)

```

```

[66]: print(f"MAE: {mae}")
      print(f"MSE: {mse}")
      print(f"R²: {r2}")

      # Store metrics in your metrics DataFrame
      metrics.loc[["mae", "mse", "r2"], f"XGBoost_{best_params_xgb['n_estimators']}"] =
      ↪ [mae, mse, r2]

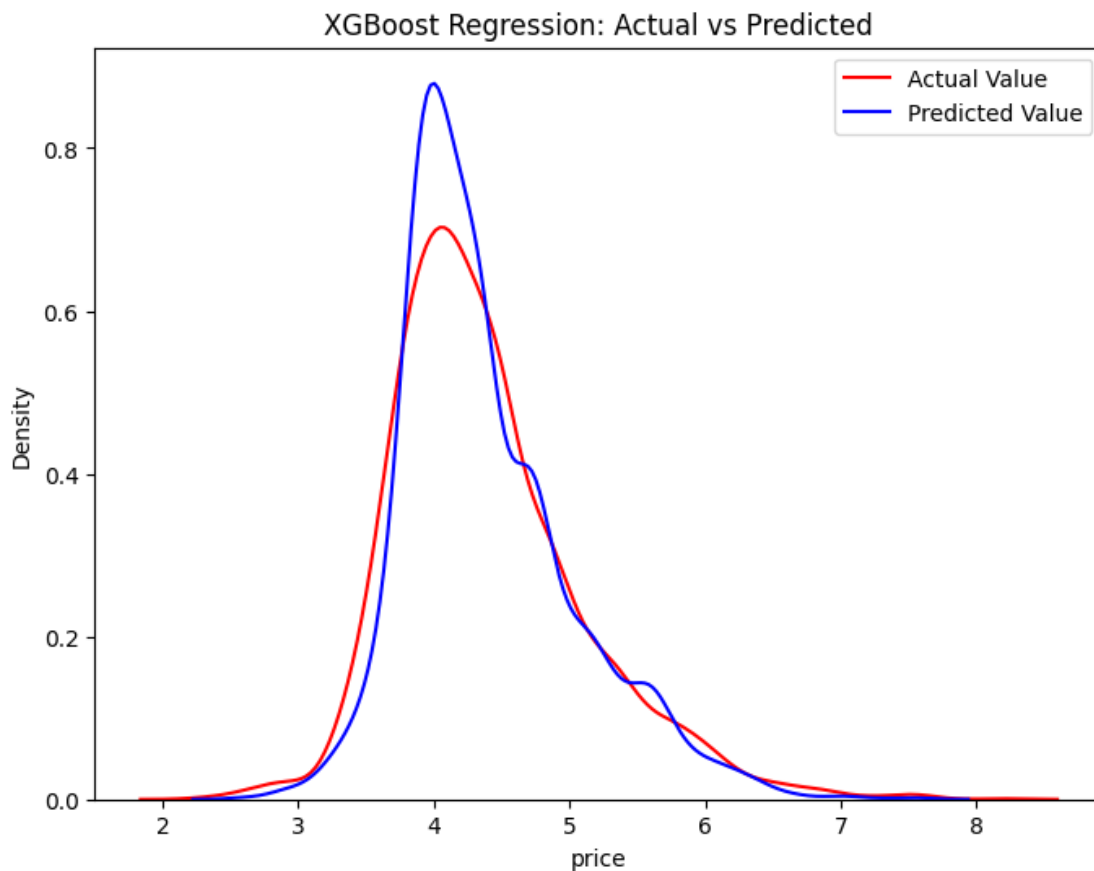
      # Plot predicted vs actual
      plt.figure(figsize=(8,6))
      sns.kdeplot(y_test, color="r", label="Actual Value")
      sns.kdeplot(y_pred_xgb, color="b", label="Predicted Value")
      plt.title("XGBoost Regression: Actual vs Predicted")
      plt.legend()
      plt.show()

```

```

MAE: 0.22848523666904708
MSE: 0.09805145900966927
R²: 0.8038258404849922

```



```
[67]: metrics
```

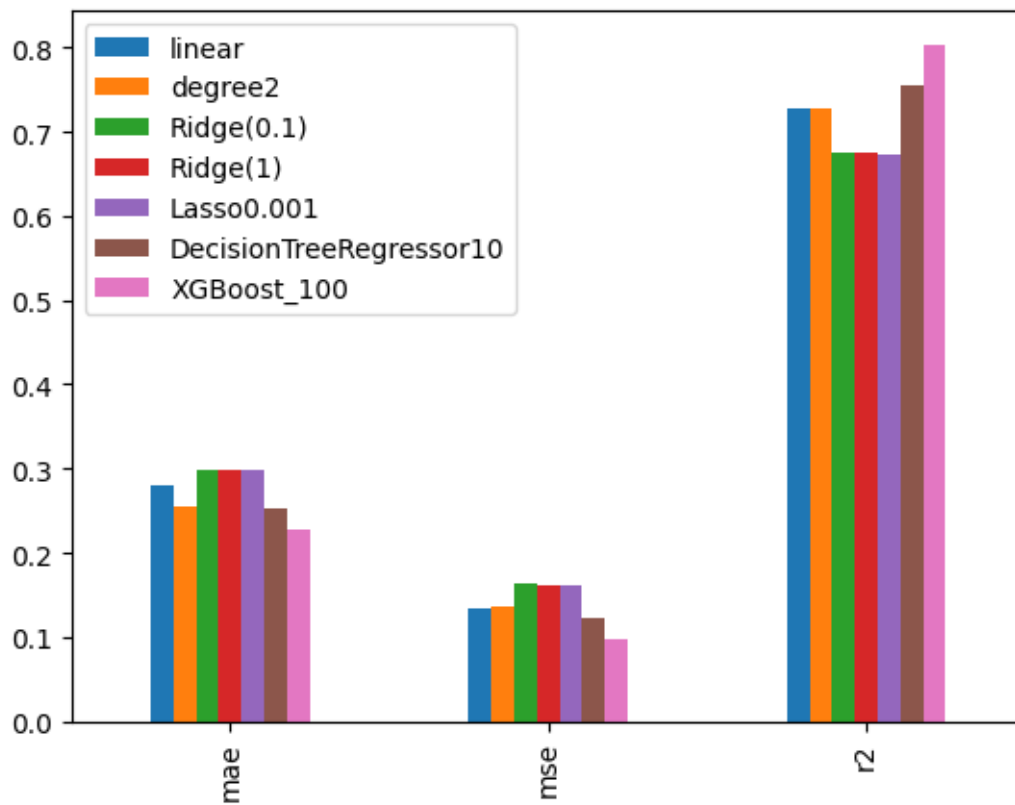
```
[67]:
```

	linear	degree2	Ridge(0.1)	Ridge(1)	Lasso0.001	\
mae	0.280212	0.254750	0.298482	0.298388	0.298388	
mse	0.134471	0.136016	0.162526	0.162428	0.162428	
r2	0.728554	0.727869	0.674830	0.675025	0.673785	

	DecisionTreeRegressor10	XGBoost_100
mae	0.253254	0.228485
mse	0.122499	0.098051
r2	0.754912	0.803826

```
[68]: metrics.plot(kind = "bar")
```

```
[68]: <Axes: >
```



```
[69]: import joblib
      joblib.dump(model, "bangalore_price_model.pkl")
```

```
[69]: ['bangalore_price_model.pkl']
```

```
[ ]:
```

```
[ ]:
```

```
[ ]:
```

```
[ ]:
```